

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN MÔN HỌC
Toán ứng dụng và thống kê**

Giáo viên hướng dẫn: ThS. Lê Nhật Nam
ThS. Võ Nam Thục Đoan

Nhóm thực hiện: Nhóm 3 - Lớp 24CTT2
Thành viên: 24120018 – Đào Thị Quỳnh Anh
24120347 - Phan Lê Đăng Khoa
24120348 – Hà Đăng Khôi
24120352 – Lương Trung Kiên
24120418 – Nguyễn Minh Quân

Hồ Chí Minh, 2026

Mục lục

Bảng phân công công việc	7
0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung	10
0.1 Tổ chức file theo hướng Module hóa	10
0.2 Module cấu hình toàn cục: config.py	11
0.3 Các hàm tiện ích nội bộ dùng chung: utils.py	12
0.4 Module tiện ích kiểm thử dùng chung: test_utils.py	15
0.5 Quy ước Import và Chạy Test	16
0.6 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon	17
0.6.1 Giới hạn biểu diễn: Machine Epsilon	17
0.6.2 Chiến lược quản lý sai số trong dự án	17
0.6.3 Ảnh hưởng của điều kiện số lên OLS	18
1 Phần 1: Lý Thuyết Data Fitting và Minh Họa	19
1.1 Bài Toán Data Fitting	19
1.1.1 Phát Biểu Bài Toán Tổng Quát	19
1.1.2 Các Giả Thiết Gauss–Markov	19
1.2 Phương Pháp Ordinary Least Squares (OLS)	20
1.2.1 Hàm Mất Mát và Nghiệm OLS	20
1.2.2 Ma Trận Chiều và Hat Matrix	21
1.2.3 Định Lý Gauss–Markov	21
1.2.4 Ước Lượng Phương Sai Nhiễu	23
1.3 Đánh Giá Mô Hình	23
1.3.1 Hệ Số Xác Định R^2 và $\bar{R}^2$ Hiệu Chỉnh	23
1.3.2 Kiểm Định Giả Thuyết	23
1.4 Các Vấn Đề Nâng Cao	24
1.4.1 Đa Cộng Tuyến và Hệ Số VIF	24
1.4.2 Hồi Quy Ridge và Lasso (Regularization)	25
1.5 Phân Tích Phần Dư	26
1.6 Cross-Validation và Lựa Chọn Mô Hình	27
1.6.1 k -Fold Cross-Validation	27
1.6.2 Tiêu Chí Lựa Chọn Mô Hình	29
1.7 Kết Quả Cài Đặt và Kiểm Chứng	30
1.7.1 Tổng Hợp Các Hàm Cài Đặt	30
1.7.2 Cấu Trúc Phụ Thuộc Giữa Các Hàm	30

1.7.3	Kiểm Chứng Với NumPy và Sklearn	31
1.7.4	Nhận Xét Kết Quả Phần 1	31
2	Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	33
2.1	Giới Thiệu	33
2.2	Hiểu Dữ Liệu (Data Understanding)	33
2.2.1	Nguồn dữ liệu và mô tả bài toán	33
2.2.2	Các biến trong dữ liệu	35
2.2.3	Khám phá dữ liệu ban đầu	35
2.2.4	Kiểm tra chất lượng dữ liệu	37
2.3	Chuẩn Bị Dữ Liệu (Data Preparation)	39
2.3.1	Lựa chọn biến (Select Data)	39
2.3.2	Làm sạch dữ liệu (Clean Data)	44
2.3.3	Xây dựng dữ liệu (Construct Data)	44
2.4	Xây Dựng Mô Hình (Modeling)	50
2.4.1	OLS full và chọn biến theo p-value	50
2.4.2	Chọn siêu tham số cho Ridge và Lasso	51
2.5	Dự Báo (Prediction)	53
2.6	Đánh Giá Mô Hình (Evaluation)	54
2.6.1	So sánh mô hình tuyến tính	54
2.6.2	Phân tích hệ số và feature importance	56
2.6.3	Phân tích phần dư và giả định mô hình	57
2.6.4	Kiểm chứng giả thuyết Giãn nở nhiệt (Thermal Expansion Hypothesis)	58
2.6.5	Phương pháp nâng cao: Kernel Ridge Regression	59
2.7	Kết Luận Phần 2	60
3	Kết luận	61
3.1	Tóm tắt kết quả đạt được	61
3.2	Bài học rút ra	62
3.3	Khó khăn chuyên môn và các nghịch lý Toán học tính toán	62

Danh sách hình vẽ

1	Phân phối $\hat{\beta}_{OLS}$ (xanh) và $\tilde{\beta}_{alt}$ (cam) qua 1000 mô phỏng. Đường đỏ dọc là giá trị β thực. OLS tập trung xung quanh β thực (không chệch) với độ phân tán thấp hơn estimator thay thế (phương sai nhỏ hơn).	22
2	Ridge Trace: giá trị các hệ số hồi quy theo λ (log scale). Khi $\lambda \rightarrow 0$, các hệ số tiệm cận nghiệm OLS; khi $\lambda \rightarrow \infty$, tất cả hệ số co về 0.	26
3	λ vs CV-MSE (5-fold). Đường đỏ đứt là λ^* tối ưu. Vùng λ quá nhỏ: mô hình overfit (MSE cao trên validation); λ quá lớn: underfit do shrinkage quá mạnh.	27
4	Bốn biểu đồ chẩn đoán phần dư. Trên dữ liệu giả lập thỏa mãn GM1–GM5: (1) phần dư phân tán đều, (2) Q-Q plot gần tuyến tính, (3) phương sai đồng đều, (4) Cook's Distance không có điểm nào vượt ngưỡng $4/n$	28
5	Sơ đồ phụ thuộc giữa các hàm Phần 1. Màu xanh là các module cơ sở (utils.py, config.py); mũi tên thể hiện hàm gọi/phụ thuộc vào hàm khác.	31
6	Histogram các biến số trước tiền xử lý. Nhiều biến có đuôi phải dài, đặc biệt là pl_orbper, pl_insol, pl_trandep và sy_dist.	36
7	Ma trận tương quan Pearson. Các nhóm biến quỹ đạo/năng lượng như pl_orbper, pl_orbsmax, pl_eqt, pl_insol có tương quan đáng chú ý, nên cần kiểm tra đa cộng tuyến bằng VIF.	37
8	Scatter plot giữa target pl_rade và các biến có tương quan cao nhất. Quan hệ giữa target và predictors có xu hướng tuyến tính cục bộ nhưng vẫn còn nhiều và ngoại lai.	37
9	So sánh phân phối trước và sau log-transform. Các biến có đuôi phải dài được nén thang đo rõ rệt, đặc biệt là pl_orbper, pl_insol và pl_bmasse.	46
10	Boxplot trước và sau Winsorization. Các điểm cực trị được kéo về ngưỡng fit trên train, trong khi phần trung tâm của phân phối được giữ nguyên.	47
11	So sánh phân phối observed và phân được MICE impute. Giá trị impute không bị gom về một điểm trung vị duy nhất mà vẫn có độ phân tán theo quan hệ đa biến.	48
12	VIF trước và sau lọc. Việc loại log_pl_orbper và log_pl_eqt đưa tất cả VIF còn lại xuống dưới ngưỡng 10.	49
13	Actual vs Predicted trên tập test của mô hình Lasso. Các điểm càng gần đường chéo thì dự báo càng sát giá trị thật.	54
14	So sánh Test R^2 , RMSE và MAE giữa các mô hình tuyến tính. Lasso đạt Test R^2 cao nhất, nhưng khác biệt giữa Lasso, OLS full và Ridge rất nhỏ.	55
15	Feature importance dựa trên hệ số Ridge sau chuẩn hóa. Vì các feature đã được standardize, độ lớn hệ số có thể so sánh trực tiếp hơn.	56

16	Bốn biểu đồ chẩn đoán phần dư của mô hình Lasso: Residuals vs Fitted, Normal Q-Q, Scale-Location và Cook's Distance.	57
----	--	----

Danh sách bảng

1	Bảng phân công công việc và mức độ đóng góp	7
2	Tổng hợp các hàm đã cài đặt trong Phần 1	30
3	Kết quả kiểm chứng F1 và F6 với NumPy/sklearn	31
4	Quy mô dữ liệu qua các bước chính	34
5	Nhóm biến ban đầu trong planet.csv	35
6	Một vài dòng dữ liệu mẫu từ planet.csv trước domain restriction	35
7	Thống kê mô tả của một số biến số sau domain restriction	36
8	Kiểm tra duplicate sau domain restriction	38
9	Các cột có missing values nhiều nhất trước tiền xử lý	38
10	Các nhóm biến bị loại trước khi tạo planet.csv	40
11	Quyết định xử lý chi tiết các nhóm cột từ raw.csv	41
12	Các cột được giữ trong planet.csv và vai trò mô hình hóa	43
13	Ảnh hưởng của log-transform lên skewness	45
14	Tóm tắt Winsorization trên tập train	47
15	Missing values sau log-transform trên tập train	47
16	Tóm tắt giá trị được MICE impute trên tập train	48
17	Lịch sử lọc VIF	49
18	VIF cuối cùng của các feature còn lại	49
19	Bảng suy diễn hệ số OLS full	51
20	Một số kết quả Ridge CV	51
21	Kết quả Lasso CV	52
22	Một số dự báo của mô hình Lasso trên tập test	53
23	So sánh các mô hình trên tập test	54
24	Kiểm định chẩn đoán trên phần dư test của Lasso	58
25	Một số cấu hình Kernel Ridge theo validation RMSE	59
26	Kết quả Kernel Ridge tốt nhất trên test set	59

Danh mục mã nguồn

1	Module <code>config.py</code> — hằng số và hàm xử lý sai số dùng chung	12
2	Trích đoạn <code>utils.py</code> : các hàm cốt lõi cho OLS	14
3	Lệnh chạy toàn bộ unit test từ thư mục gốc	16
4	Hàm <code>ols_fit</code> - tính nghiệm OLS từ công thức Normal Equations	20
5	Hàm <code>coef_inference</code> - tính standard error, t-statistic, p-value và khoảng tin cậy 95%	24
6	Hàm <code>vif</code> - tính VIF cho từng biến, dùng F1 và F3	25
7	Hàm <code>kfold_cv</code> - thực hiện k-fold cross-validation, dùng được với <code>ols_fit</code> , <code>ridge_fit</code> , <code>lasso_fit</code>	28
8	Tách train/test trước khi fit preprocessing	44

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ MỨC ĐỘ ĐÓNG GÓP

Bảng 1: Bảng phân công công việc và mức độ đóng góp

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
1	24120018 Đào Thị Quỳnh Anh	• Phần 1: Cài đặt thuật toán hồi quy có cực tiểu hóa hình phạt: Ridge Regression (nghiệm dạng đóng) và Lasso Regression (giải bằng phương pháp Coordinate Descent). • Phần 1: Xây dựng hệ thống chẩn đoán mô hình trực quan (vẽ 4 biểu đồ phần dư) và cài đặt thuật toán kiểm chứng chéo K-Fold Cross-Validation từ đầu để tinh chỉnh siêu tham số λ. • Phần 2 (T7 & QA): Cài đặt mô hình nâng cao (Kernel Ridge Regression hoặc Bayesian Linear Regression) để so sánh đối chiếu. Thực hiện rà soát chất lượng mã nguồn (Quality Assurance), kiểm tra Unit Test và đảm bảo tính nhất quán của hằng số RANDOM_STATE.	100%
2	24120347 Phan Lê Đăng Khoa	• Phần 1: Lập trình lõi thuật toán Hồi quy tuyến tính đa biến (OLS) bằng phương trình chuẩn (Normal Equations) sử dụng Python thuần. • Phần 1: Tính toán ma trận chiếu (Hat Matrix), kiểm chứng tính chất đại số tuyến tính (lũy đẳng, đối xứng) và trích xuất trị riêng. Cài đặt hệ thống đo lường với các chỉ số thống kê cốt lõi (R^2, Adj-R^2, RSS, F-statistic). • Phần 2 (T3): Xây dựng lớp DataPipeline hướng đối tượng nhằm tự động hóa các khâu: nội suy KNN, xử lý ngoại lai (Winsorization), chuẩn hóa (Scaling) và mã hóa (Encoding), đảm bảo tuyệt đối không xảy ra rò rỉ dữ liệu (No Data Leakage).	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
3	24120348 Hà Đăng Khôi	• Cơ sở hạ tầng: Khởi tạo kiến trúc dự án (README.md, requirements.txt) và thiết kế Jupyter Notebook trình bày cơ sở lý thuyết Toán học cho các thuật toán từ F1 đến F5. • Phần 2 (T1, T2): Chọn lọc và tải tập dữ liệu thực tế. Thực hiện Khảo sát dữ liệu toàn diện (EDA): báo cáo dữ liệu khuyết thiếu, phân tích phân phối (Histogram, Boxplot), và tính toán ma trận tương quan (Heatmap). • Phần 2 (T6): Xây dựng thuật toán nhận diện điểm ngoại lai bằng phương pháp IQR. Trực quan hóa mức độ quan trọng của các biến độc lập (Feature Importance) bằng Bar chart và tổng hợp báo cáo Notebook.	100%
4	24120352 Lương Trung Kiên	• Cơ sở hạ tầng: Thiết lập khung báo cáo $\text{\LaTeX}$ chuẩn học thuật. Thiết kế Jupyter Notebook minh họa lý thuyết phần chuẩn đoán và xác thực chéo (các hàm từ F6 đến F10). • Phần 2 (T5): Lập bảng tổng hợp, so sánh hiệu năng các mô hình trên tập kiểm thử thông qua độ đo MAE, RMSE, R^2. Phân tích chuyên sâu biểu đồ phần dư của mô hình tối ưu. • Báo cáo: Đưa ra các nhận định đánh giá giả định Gauss-Markov dựa trên số liệu. Chịu trách nhiệm viết phần Kết luận, căn chỉnh định dạng $\text{\LaTeX}$, xử lý caption hình ảnh và biên soạn tài liệu tham khảo theo chuẩn BibTeX.	100%

STT	Họ tên / MSSV	Nhiệm vụ đảm nhiệm chi tiết	Đóng góp
5	24120418 Nguyễn Minh Quân	• Phần 1: Cài đặt module suy diễn thống kê: tính toán sai số chuẩn, t-statistic, p-value và khoảng tin cậy 95% cho các hệ số hồi quy. • Phần 1: Cài đặt thuật toán tính Hệ số nhân phương sai (VIF) nhằm phát hiện đa cộng tuyến. Lập trình mô phỏng Monte Carlo ($N = 1000$) để kiểm chứng thực nghiệm định lý Gauss-Markov. • Phần 2 (T4): Sử dụng Pipeline để huấn luyện và đánh giá 4 mô hình (OLS cơ bản, OLS chọn lọc biến, Ridge, Lasso) trên tập dữ liệu thực.	100%

0 Tổng quan Kiến trúc Hệ thống và Các Module dùng chung

Trước khi đi vào chi tiết của từng phần, mục này trình bày kiến trúc tổng thể của toàn bộ dự án — cách các module Python được tổ chức, các thành phần hạ tầng dùng chung giữa Phần 1 (lý thuyết và minh họa) và Phần 2 (ứng dụng thực tế), và nền tảng thiết kế về quản lý sai số số học mà toàn bộ codebase đều tuân theo. Việc hiểu rõ kiến trúc này giúp tránh trùng lặp mã, đảm bảo tính nhất quán khi kiểm chứng kết quả, và tạo điều kiện để hai phần của dự án chia sẻ cùng một pipeline hạ tầng.

0.1 Tổ chức file theo hướng Module hóa

Toàn bộ dự án được thiết kế theo nguyên tắc **module hóa** (modular design): mỗi file Python đảm nhiệm một trách nhiệm duy nhất và rõ ràng, hạn chế phụ thuộc vòng (circular dependency), đồng thời cho phép tái sử dụng các thành phần cốt lõi mà không phải viết lại mã. Cụ thể, dự án được chia thành bốn nhóm chức năng lớn:

- **Hạ tầng dùng chung (root):** `config.py` và `utils.py` nằm ở thư mục gốc. Đây là các module không phụ thuộc vào bất cứ module nào khác trong dự án; mọi module cấp trên đều có thể import từ đây. Hai module này cung cấp hằng số toàn cục, phép toán đại số tuyến tính từ đầu, và hàm xây dựng null vector phục vụ demo Gauss–Markov.
- **Tiện ích kiểm thử dùng chung:** `test_utils.py` cũng nằm ở thư mục gốc, cung cấp các hàm sinh dữ liệu giả lập (`make_linear_data`, `make_collinear_data`), các hàm assert có log màu (`assert_close`, `assert_shape`, v.v.) và các hàm kiểm chứng với sklearn (`verify_vs_sklearn_ols`, `verify_vs_sklearn_ridge`). Tất cả file unit test trong dự án import từ đây.
- **Module nghiệp vụ Phần 1 (part1/):** Gồm các file cài đặt thuật toán OLS và các phương pháp liên quan — `ols_implementation.py` (F1–F5), `ridge_lasso.py` (F6–F7), `residual_analysis.py` (F8), `cross_validation.py` (F9), `gauss_markov_demo.py` (F10) — cùng file kiểm thử riêng `test_ols_implementation.py`.
- **Module nghiệp vụ Phần 2 (part2/):** Gồm `data_pipeline.py` (pipeline tiền xử lý dữ liệu thực), `model_comparison.py` (so sánh ≥ 3 mô hình trên test set), và `advanced_methods.py` (Kernel/Bayesian, tùy chọn). Các module này tái sử dụng trực tiếp F1, F6, F7, F8, F9 từ part1/ thay vì viết lại.

Cấu trúc thư mục đầy đủ của dự án như sau:

```

Project2_DataFitting/
|-- config.py           # Hằng số toàn cục (EPSILON, RANDOM_STATE)
|-- utils.py           # Phép toán đại số tuyến tính tu đầu
|-- test_utils.py      # Tiện ích kiểm thử dùng chung
|-- part1/
|   |-- ols_implementation.py  # F1-F5: OLS, Hat matrix, VIF, ...
|   |-- ridge_lasso.py        # F6-F7: Ridge, Lasso, Trace
|   |-- residual_analysis.py  # F8: 4 biểu đồ chẩn đoán
|   |-- cross_validation.py   # F9: k-Fold CV
|   |-- gauss_markov_demo.py  # F10: Monte Carlo demo
|   '-- test_ols_implementation.py
|-- part2/
|   |-- data/
|   |   '-- <ten_dataset>.csv
|   |-- data_pipeline.py      # Tiền xử lý, encoding, chuẩn hóa
|   |-- model_comparison.py   # So sánh mô hình trên test set
|   '-- advanced_methods.py   # Kernel / Bayesian (tùy chọn)
|-- report/
|   '-- report.pdf
'-- requirements.txt

```

Cấu trúc này đảm bảo: khi một module lỗi (ví dụ `utils.py`) được cập nhật, toàn bộ codebase hưởng ngay lợi ích mà không cần sửa lại ở nhiều nơi. Đồng thời, Phần 2 không cần cài đặt lại OLS hay Ridge — các hàm từ Phần 1 được import trực tiếp, đảm bảo kết quả nhất quán giữa hai phần.

0.2 Module cấu hình toàn cục: `config.py`

Module `config.py` là file cấu hình dùng chung đầu tiên cần được hiểu. Nó định nghĩa các hằng số và hàm tiện ích được sử dụng xuyên suốt toàn bộ dự án, từ các phép tính đại số trong `utils.py` đến quá trình shuffle trong `cross_validation.py`.

- `EPSILON = 1e-12`: Ngưỡng epsilon toàn cục. Mọi số có giá trị tuyệt đối nhỏ hơn ngưỡng này đều được coi là bằng 0 về mặt số học. Ngưỡng này được dùng nhất quán trong tất cả phép chia, kiểm tra pivot, và tính VIF để tránh chia cho 0.
- `RANDOM_STATE = 42`: Seed toàn cục đảm bảo tính tái lập (*reproducibility*) của toàn bộ dự án — từ sinh dữ liệu giả lập trong `test_utils.py`, Fisher-Yates shuffle trong `cross_validation.py`, đến LCG random generator trong `gauss_markov_demo.py`.
- `is_zero(x)`: Kiểm tra xem một số x có xấp xỉ bằng 0 hay không theo ngưỡng `EPSILON`.
- `zero_rectify(value)`: Nếu `value` đủ nhỏ thì trả về 0.0 thay vì giữ nguyên giá trị nhiễu số.

học (ví dụ: $-2.77e-16$), tránh hiện tượng “ô nhiễm” kết quả qua nhiều bước tính toán.

- `calculate_relative_error(A, x_hat, b)`: Tính sai số tương đối $\|A\hat{x} - b\|_2 / \|b\|_2$ — dùng trong unit test để kiểm tra chất lượng nghiệm OLS và Ridge.

```

1 import math
2
3 EPSILON: float = 1e-12
4 RANDOM_STATE: int = 42
5
6 def is_zero(x: float) -> bool:
7     """Kiểm tra một số có xấp xỉ bằng 0 hay không."""
8     return abs(x) < EPSILON
9
10 def zero_rectify(value: float) -> float:
11     """Khu giá trị về 0 nếu nó đủ nhỏ."""
12     return 0.0 if is_zero(value) else value
13
14 def calculate_relative_error(A, x_hat, b) -> float:
15     """Tính sai số tương đối ||A*x_hat - b||_2 / ||b||_2."""
16     n = len(b)
17     residual = [sum(A[i][j]*x_hat[j] for j in range(len(x_hat)))-b[i]
18                 for i in range(n)]
19     norm_r = math.sqrt(sum(r*r for r in residual))
20     norm_b = math.sqrt(sum(bi*bi for bi in b))
21     if is_zero(norm_b):
22         return 0.0 if is_zero(norm_r) else float('inf')
23     return norm_r / norm_b

```

Đoạn mã 1: Module `config.py` — hằng số và hàm xử lý sai số dùng chung

Việc tách riêng hằng số và hàm xử lý sai số vào `config.py` giúp đảm bảo tính nhất quán: giá trị ngưỡng `EPSILON` chỉ cần khai báo một lần duy nhất. Đặc biệt với dự án này, `RANDOM_STATE` đóng vai trò quan trọng hơn so với các dự án tính toán số thuần túy, vì kết quả cross-validation và demo Gauss–Markov phụ thuộc trực tiếp vào thứ tự ngẫu nhiên của dữ liệu.

0.3 Các hàm tiện ích nội bộ dùng chung: `utils.py`

Module `utils.py` cung cấp một thư viện các phép toán đại số tuyến tính cơ bản được tái sử dụng xuyên suốt dự án. Toàn bộ được cài đặt **từ đầu bằng Python thuần**, không phụ thuộc vào `numpy` hay bất kỳ thư viện số học nào. Đây là yêu cầu bắt buộc của đề bài — các hàm này là “động cơ” của toàn bộ cài đặt OLS, Ridge, Lasso, và Hat Matrix.

Nhóm phép toán ma trận cơ bản

- `identity_matrix(n)`: Tạo ma trận đơn vị I_n dưới dạng list 2 chiều.

- `transpose(A)`: Chuyển vị ma trận bằng `zip(*A)`, trả về $\mathbf{A}^T$. Được dùng trong mọi lần tính $\mathbf{X}^T \mathbf{X}$ và $\mathbf{X}^T \mathbf{y}$.
- `matmul(A, B)`: Nhân hai ma trận theo thuật toán $\mathcal{O}(mnp)$. Có tối ưu nhỏ: bỏ qua vòng lặp trong nếu $a_{ik} \approx 0$ (kiểm tra qua `is_zero`), giảm phép tính không cần thiết trên ma trận thưa.
- `matvec(A, v)`: Tích ma trận–vector $\mathbf{A}\mathbf{v}$, trả về list 1 chiều. Được dùng để tính $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$ trong mọi hàm fit.
- `dot_product(u, v)`: Tích vô hướng $\mathbf{u}^T \mathbf{v}$. Được dùng trong Gram–Schmidt và coordinate descent của Lasso.
- `vector_norm(v)`: Chuẩn Euclidean $\|\mathbf{v}\|_2$; dùng `max(0.0, ...)` để phòng kết quả âm nhỏ do sai số số học trước khi lấy căn.
- `normalize(v)`: Chuẩn hóa vector về đơn vị ($\|\mathbf{v}\|_2 = 1$); trả về vector $\mathbf{0}$ nếu chuẩn ban đầu xấp xỉ 0.
- `add_bias(X)`: Thêm cột hằng số 1 vào đầu ma trận $\mathbf{X}$ tạo ra ma trận design $[\mathbf{1} \mid \mathbf{X}]$. Được gọi đầu tiên trong `ols_fit`, `hat_matrix`, `coef_inference` và các hàm inference khác.

Nhóm giải hệ tuyến tính

- `solve_system(A, b)`: Giải hệ $\mathbf{Ax} = \mathbf{b}$ bằng phép khử Gauss với *partial pivoting* và back-substitution. Ném `ValueError` khi ma trận suy biến ($\text{pivot} < 10^{-14}$). Đây là hàm cốt lõi nhất — được gọi bởi `ols_fit` (giải Normal Equations) và `ridge_fit` (giải hệ Ridge đã điều chỉnh).
- `inverse(A)`: Tính $\mathbf{A}^{-1}$ bằng phương pháp Gauss–Jordan trên ma trận mở rộng $[\mathbf{A} \mid \mathbf{I}]$. Được dùng trong `hat_matrix` (tính $(\mathbf{X}^T \mathbf{X})^{-1}$) và `coef_inference` (tính ma trận hiệp phương sai $\hat{\sigma}^2(\mathbf{X}^T \mathbf{X})^{-1}$).

Nhóm kiểm tra và chuẩn hóa

- `is_upper_triangular(U, tol)`: Kiểm tra ma trận $\mathbf{U}$ có phải tam giác trên không.
- `check_identity(M, tol)`: Kiểm tra ma trận $\mathbf{M}$ có xấp xỉ ma trận đơn vị không. Dùng trong unit test để kiểm chứng $(\mathbf{X}^T \mathbf{X})(\mathbf{X}^T \mathbf{X})^{-1} \approx \mathbf{I}$.
- `rectify_matrix(M)`, `rectify_vector(v)`: Áp dụng `zero_rectify` lên từng phần tử. Dùng sau các phép nhân ma trận lớn để làm sạch nhiễu số học tích lũy.
- `max_abs_diff(A, B)`: Tính $\max_{ij} |A_{ij} - B_{ij}|$ — thước đo độ chính xác trong unit test, ví dụ kiểm tra $\mathbf{H}^2 \approx \mathbf{H}$ (tính idempotent của Hat Matrix).

Nhóm trực giao hóa và null space

- `orthogonalize(v, basis)`: Chiếu vector $\mathbf{v}$ ra khỏi tất cả các vector trong `basis` theo công thức Gram–Schmidt cổ điển:

$$\mathbf{w} = \mathbf{v} - \sum_{\mathbf{b} \in \text{basis}} \langle \mathbf{v}, \mathbf{b} \rangle \cdot \mathbf{b}. \quad (1)$$

Mỗi lần trừ xong một thành phần, kết quả được làm sạch qua `zero_rectify` để triệt tiêu nhiễu dấu phẩy động.

- `find_new_unit_vector(basis, dim)`: Khi cần bổ sung vector mới vào cơ sở trực giao, hàm lần lượt thử các vector đơn vị $\mathbf{e}_1, \mathbf{e}_2, \dots$. Mỗi ứng viên được trực giao hóa với basis rồi chuẩn hóa. Ném `ValueError` nếu thất bại.
- `build_null_vector(X_bias)`: Xây dựng vector $\mathbf{v} \in \text{Null}(\mathbf{X}^T)$, tức $\mathbf{X}^T \mathbf{v} \approx \mathbf{0}$, bằng cách tạo vector ngẫu nhiên (LCG reproducible) rồi chiếu ra khỏi không gian cột của $\mathbf{X}$. Được dùng duy nhất bởi `gauss_markov_demo.py` để xây dựng estimator thay thế chứng minh OLS là BLUE.

```

1 from config import is_zero, zero_rectify, EPSILON
2 import math
3
4 def add_bias(X: list[list[float]]) -> list[list[float]]:
5     """Thêm cột 1 (bias/intercept) vào đầu ma trận X."""
6     return [[1.0] + list(row) for row in X]
7
8 def solve_system(A, b) -> list[float]:
9     """Giải Ax = b bằng khu Gauss với partial pivoting."""
10    n = len(b)
11    M = [A[i][:] + [b[i]] for i in range(n)]
12    for col in range(n):
13        # Partial pivoting
14        pivot_row = max(range(col, n), key=lambda r: abs(M[r][col]))
15        M[col], M[pivot_row] = M[pivot_row], M[col]
16        if abs(M[col][col]) < 1e-14:
17            raise ValueError("Ma trận suy biến.")
18        for row in range(col + 1, n):
19            factor = M[row][col] / M[col][col]
20            for j in range(col, n + 1):
21                M[row][j] -= factor * M[col][j]
22    # Back-substitution
23    x = [0.0] * n
24    for i in range(n - 1, -1, -1):
25        x[i] = M[i][n]
26        for j in range(i + 1, n):
27            x[i] -= M[i][j] * x[j]
28        x[i] /= M[i][i]
29    return x
30
31 def build_null_vector(X_bias: list[list[float]]) -> list[float]:
32     """Xây dựng v thuộc Null(X^T): X^T v ≈ 0 (dùng cho GM demo)."""
33    n, p1 = len(X_bias), len(X_bias[0])
34    cols = [[X_bias[i][j] for i in range(n)] for j in range(p1)]
35    state = 12345

```

```

36     v = []
37     for _ in range(n):
38         state = (state * 1103515245 + 12345) & 0x7FFFFFFF
39         v.append((state / 0x7FFFFFFF) * 2.0 - 1.0)
40     for col in cols:                # Gram-Schmidt ra khỏi span(X)
41         c = dot_product(v, col) / max(dot_product(col, col), EPSILON)
42         v = [v[i] - c * col[i] for i in range(n)]
43     norm_v = math.sqrt(sum(vi*vi for vi in v))
44     return [vi / norm_v for vi in v]

```

Đoạn mã 2: Trích đoạn `utils.py`: các hàm cốt lõi cho OLS

0.4 Module tiện ích kiểm thử dùng chung: `test_utils.py`

Một điểm khác biệt đáng chú ý của dự án này so với dự án trước là sự xuất hiện của `test_utils.py` — một module kiểm thử dùng chung hoàn chỉnh. Module này phục vụ ba mục đích:

Sinh dữ liệu giả lập nhất quán

- `make_linear_data(n, beta, sigma, seed)`: Sinh dataset tuyến tính $y = X\beta + \varepsilon$ với $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, dùng `numpy.default_rng(seed)` để tái lập. Trả về (X, y) với X chưa có cột bias.
- `make_multifeature_data(n, p, sigma, seed)`: Sinh dataset nhiều biến với β ngẫu nhiên — dùng để kiểm tra tổng quát hơn.
- `make_collinear_data(n, seed)`: Sinh dataset có đa cộng tuyến: $x_3 \approx x_1 + x_2 + \varepsilon_{\text{small}}$. Dùng đặc biệt cho unit test F5 (vif) để xác nhận $VIF > 10$ được phát hiện đúng.

Toàn bộ file unit test của dự án đều dùng các factory này thay vì tự sinh data riêng. Điều này đảm bảo cùng một seed cho ra cùng một dữ liệu, giúp debug và so sánh kết quả giữa các thành viên nhóm.

Hàm assert có log màu Các hàm `assert_*` trong `test_utils.py` không ném exception mà trả về bool và in log màu qua `TestLogger`. Điều này cho phép chạy toàn bộ suite mà không dừng ở test đầu tiên thất bại:

- `assert_close(actual, expected, label, rtol, atol)`: So sánh hai giá trị/array với dung sai tương đối và tuyệt đối, dùng `numpy.testing.assert_allclose` nội bộ.
- `assert_equal(actual, expected, label)`: So sánh bằng tuyệt đối (`==`).
- `assert_true(condition, label, details)`: Kiểm tra điều kiện Boolean.
- `assert_shape(arr, expected_shape, label)`: Kiểm tra shape của array/list qua `numpy.asarray().shape`.
- `assert_in_range(val, lo, hi, label)`: Kiểm tra $lo \leq val \leq hi$.
- `assert_raises(exc_type, fn, *args, label)`: Kiểm tra hàm `fn` ném đúng loại exception. Phân biệt giữa “không ném exception” và “ném sai loại exception” — cả hai đều được

log riêng.

Hàm kiểm chứng với sklearn

- `verify_vs_sklearn_ols(X, y, beta_hat, rtol)`: So sánh `beta_hat` từ F1 với `sklearn.LinearRegression` — kiểm tra cả intercept lẫn coefficients.
- `verify_vs_sklearn_ridge(X, y, beta_hat, lam, rtol)`: So sánh với `sklearn.Ridge(alpha=lam)`. Dùng ngưỡng `rtol=1e-3` (lỏng hơn OLS) vì Ridge có chuẩn hóa có thể khác convention.
Cả hai hàm đều bỏ qua kiểm tra (trả về True với cảnh báo) nếu sklearn không được cài đặt, đảm bảo unit test vẫn chạy được trên môi trường không có sklearn.

0.5 Quy ước Import và Chạy Test

Để tránh lỗi import path giữa các môi trường khác nhau, dự án áp dụng quy ước thống nhất:

Quy tắc import path toàn dự án

- Tất cả file trong `part1/` import lẫn nhau không có prefix `part1..`

Dung - chạy tu thu mục `part1/`
`from ols_implementation import ols_fit, hat_matrix`
- Mỗi file đặt `sys.path.insert(0, os.path.dirname(__file__))` ở đầu để đảm bảo import đúng dù được gọi từ thư mục nào.
- Phần 2 import từ Phần 1 qua `sys.path.insert` trở đến thư mục `part1/`:

Trong `part2/model_comparison.py`
`sys.path.insert(0, os.path.join(os.path.dirname(__file__), '../part1'))`
`from ols_implementation import ols_fit`

Chạy unit test: Tất cả file có thể chạy trực tiếp như script độc lập. Cách chạy toàn bộ từ thư mục gốc:

```
1 # Chạy tung file tu part1/
2 cd part1
3 python ols_implementation.py
4 python ridge_lasso.py
5 python residual_analysis.py
6 python cross_validation.py
7 python gauss_markov_demo.py
8
9 # Chạy file test riêng biệt
```

```
10 python test_ols_implementation.py
```

Đoạn mã 3: Lệnh chạy toàn bộ unit test từ thư mục gốc

Kết quả đầu ra được in theo định dạng nhất quán của TestLogger: mỗi test một dòng với trạng thái **[OK] PASSED** hoặc **[FAIL] FAILED**, kèm chi tiết sai số; cuối suite in thành tiến trình và tổng kết X/Y PASSED.

0.6 Quản lý Sai số Dấu phẩy động: IEEE 754 và Machine Epsilon

Toàn bộ dự án làm việc với số thực dưới dạng số dấu phẩy động 64-bit (double precision) theo chuẩn **IEEE 754**. Việc hiểu và kiểm soát sai số từ biểu diễn này là nền tảng bắt buộc — đặc biệt với dự án Data Fitting, nơi sai số số học trong quá trình giải Normal Equations có thể ảnh hưởng trực tiếp đến chất lượng ước lượng β .

0.6.1 Giới hạn biểu diễn: Machine Epsilon

Chuẩn IEEE 754 double precision biểu diễn số thực dưới dạng:

$$x = (-1)^s \cdot m \cdot 2^e, \quad (2)$$

trong đó $s \in \{0, 1\}$ là bit dấu, m là phần định trị (mantissa) 52 bit, và e là số mũ 11 bit. Hệ quả trực tiếp là không phải mọi số thực đều có thể được biểu diễn chính xác. Giữa hai số thực liên kề bất kỳ luôn tồn tại một khoảng cách nhỏ nhất, gọi là **machine epsilon** ϵ_{mach} :

$$\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}. \quad (3)$$

Đây là sai số tương đối nhỏ nhất mà phép tính dấu phẩy động có thể gây ra trong *một phép toán đơn lẻ*. Trong OLS, khi tính $(\mathbf{X}^T \mathbf{X})^{-1}$ qua nhiều bước khử Gauss, sai số tích lũy và có thể khuếch đại nếu ma trận gần suy biến (số điều kiện lớn).

0.6.2 Chiến lược quản lý sai số trong dự án

Dự án áp dụng ba lớp kiểm soát sai số chồng lên nhau:

Lớp 1 — Ngưỡng epsilon toàn cục (EPSILON = 1e-12) Thay vì so sánh bằng tuyệt đối ($x == 0.0$), mọi phép so sánh với 0 đều đi qua `is_zero(x)` với ngưỡng $\epsilon = 10^{-12}$. Ngưỡng này được chọn cẩn thận:

- Lớn hơn $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ đủ nhiều để hấp thu nhiễu số học tích lũy qua nhiều bước.
- Nhỏ hơn 10^{-6} đủ để không nhầm các giá trị nhỏ nhưng thực sự khác không (ví dụ: hệ số Lasso sau shrinkage) với số 0.

Ứng dụng cụ thể: trong `lasso_fit`, điều kiện $|z_j| > \varepsilon$ trước phép chia $\beta_j = S(\rho_j, \lambda)/z_j$ tránh chia cho 0 khi cột j là vector không.

Lớp 2 — Làm sạch kết quả trung gian (zero_rectify) Sau mỗi phép toán lớn (một bước khử Gauss, một vòng lặp coordinate descent, một lần Gram–Schmidt), kết quả được lọc qua `zero_rectify`. Điều này ngăn các số dư như $-2.77\text{e-}16$ làm “ô nhiễm” các bước tính toán tiếp theo — đặc biệt quan trọng trong `build_null_vector` khi chiếu vector ra khỏi không gian cột của $\mathbf{X}$.

Lớp 3 — Chọn chốt từng phần (Partial Pivoting) Trong `solve_system` và `inverse`, tại mỗi bước khử, hàng có $|a_{ij}|$ lớn nhất được chọn làm pivot. Điều này đảm bảo hệ số nhân $l_{ik} = a_{ik}/a_{kk}$ luôn ≤ 1 về trị tuyệt đối, hạn chế tối đa sự khuếch đại sai số qua từng bước khử. Đây là biện pháp phòng ngừa ở cấp độ thuật toán, bổ sung cho hai lớp làm sạch số học ở trên.

0.6.3 Ảnh hưởng của điều kiện số lên OLS

Một vấn đề đặc thù của Data Fitting là **số điều kiện** (condition number) của ma trận $\mathbf{X}^T \mathbf{X}$. Khi các cột của $\mathbf{X}$ có tương quan cao (đa cộng tuyến), $\kappa(\mathbf{X}^T \mathbf{X})$ trở nên rất lớn và sai số trong $\hat{\beta}$ khuếch đại tương ứng:

$$\frac{\|\delta \hat{\beta}\|}{\|\hat{\beta}\|} \leq \kappa(\mathbf{X}^T \mathbf{X}) \cdot \frac{\|\delta \mathbf{y}\|}{\|\mathbf{y}\|}. \quad (4)$$

Đây chính là lý do Ridge Regression thêm $\lambda \mathbf{I}$ vào $\mathbf{X}^T \mathbf{X}$ — không chỉ để regularize mà còn để cải thiện số điều kiện: $\kappa(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) < \kappa(\mathbf{X}^T \mathbf{X})$ với mọi $\lambda > 0$.

Trong thực nghiệm, sai số tương đối $\|\hat{\beta}_{\text{OLS}} - \beta_{\text{true}}\|/\|\beta_{\text{true}}\|$ trên dữ liệu giả lập với $\sigma = 0$ thường nằm trong khoảng 10^{-9} đến 10^{-7} — tức nhỏ hơn nhiều so với $\sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$, xác nhận rằng cài đặt Normal Equations qua `solve_system` là đủ chính xác cho mục đích của dự án.

1 Phần 1: Lý Thuyết Data Fitting và Minh Họa

Phần này trình bày nền tảng toán học của bài toán Data Fitting và phương pháp Ordinary Least Squares (OLS), kèm theo cài đặt Python từ đầu để minh họa và kiểm chứng từng kết quả lý thuyết.

1.1 Bài Toán Data Fitting

1.1.1 Phát Biểu Bài Toán Tổng Quát

Định nghĩa 1.1 - Bài Toán Data Fitting

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Bài toán Data Fitting là tìm hàm $f: \mathbb{R}^p \rightarrow \mathbb{R}$ trong một lớp hàm cho trước sao cho f xấp xỉ tốt nhất ánh xạ từ $\mathbf{x}_i$ đến y_i theo một tiêu chí đã định.

Trong mô hình **hồi quy tuyến tính**, ta giả thiết quan hệ tuyến tính giữa các biến độc lập và biến phụ thuộc:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i, \quad (5)$$

với $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là nhiễu ngẫu nhiên.

Viết dưới dạng ma trận, với $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ là *ma trận design* (cột đầu toàn 1 đại diện cho intercept):

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad (6)$$

trong đó $\mathbf{y} \in \mathbb{R}^n$ là vector quan sát và $\boldsymbol{\varepsilon} \in \mathbb{R}^n$ là vector nhiễu.

1.1.2 Các Giả Thiết Gauss–Markov

Để đảm bảo OLS có các tính chất tốt, ta cần các giả thiết Gauss–Markov (GM1–GM5):

GM1. Tuyến tính: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$.

GM2. Không hoàn hảo đa cộng tuyến: $\text{rank}(\mathbf{X}) = p + 1$ (các cột của $\mathbf{X}$ độc lập tuyến tính, đảm bảo $\mathbf{X}^T \mathbf{X}$ khả nghịch).

GM3. Ngoại sinh: $E[\boldsymbol{\varepsilon} | \mathbf{X}] = \mathbf{0}$, tức $E[\varepsilon_i | \mathbf{x}_i] = 0$ với mọi i .

GM4. Đồng phương sai (Homoscedasticity): $\text{Var}(\boldsymbol{\varepsilon} | \mathbf{X}) = \sigma^2 \mathbf{I}_n$, tức các sai số có cùng phương sai σ^2 và không tương quan với nhau.

GM5. Phân phối chuẩn (tùy chọn): $\boldsymbol{\varepsilon} | \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$ (cần cho kiểm định giả thuyết).

1.2 Phương Pháp Ordinary Least Squares (OLS)

1.2.1 Hàm Mất Mát và Nghiệm OLS

OLS tìm $\hat{\beta}$ tối thiểu hóa *Tổng Bình Phương Phần Dư* (Residual Sum of Squares - RSS):

$$\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2. \quad (7)$$

Định lý 1.1 - Nghiệm OLS (Normal Equations)

Nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch, nghiệm OLS duy nhất là:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (8)$$

Chứng minh. Lấy đạo hàm của RSS theo β :

$$\begin{aligned} \nabla_{\beta} \text{RSS} &= \nabla_{\beta} [(\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)] \\ &= -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta). \end{aligned} \quad (9)$$

Cho đạo hàm bằng 0: $\mathbf{X}^T \mathbf{X}\beta = \mathbf{X}^T \mathbf{y}$ (phương trình pháp tuyến - *Normal Equations*). Vì $\mathbf{X}^T \mathbf{X}$ khả nghịch (theo GM2), suy ra $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Để kiểm tra đây là cực tiểu (không phải cực đại), ta kiểm tra đạo hàm bậc 2: $\nabla_{\beta}^2 \text{RSS} = 2\mathbf{X}^T \mathbf{X}$ là ma trận xác định dương (positive definite) vì $\text{rank}(\mathbf{X}) = p + 1$. $\square$

Cài đặt Python - F1: ols_fit:

```

1 def ols_fit(X: list[list[float]], y: list[float]) -> dict:
2     """F1: Tính beta_hat = (X^T X)^{-1} X^T y và sigma2_hat."""
3     n, p = len(X), len(X[0])
4     X_b = add_bias(X) # them cot 1 (intercept)
5     Xt = transpose(X_b) # utils.py
6     XtX = matmul(Xt, X_b) # utils.py
7     Xty = matvec(Xt, y) # utils.py
8     beta_hat = solve_system(XtX, Xty) # utils.py (Gauss elim.)
9     y_hat = matvec(X_b, beta_hat)
10    residuals = [y[i] - y_hat[i] for i in range(n)]
11    rss = sum(r * r for r in residuals)
12    sigma2 = rss / max(n - p - 1, 1) # ung luong khong chech
13    return {"beta_hat": beta_hat, "sigma2_hat": sigma2,
14            "y_hat": y_hat, "residuals": residuals}

```

Đoạn mã 4: Hàm ols_fit - tính nghiệm OLS từ công thức Normal Equations

Hàm solve_system trong utils.py giải hệ $\mathbf{Ax} = \mathbf{b}$ bằng phép khử Gauss với partial pivoting, tránh sử dụng numpy.linalg.solve đúng theo yêu cầu đề bài.

1.2.2 Ma Trận Chiếu và Hat Matrix

Định nghĩa 1.2 - Hat Matrix

Ma trận chiếu (projection matrix) hay *Hat Matrix* được định nghĩa:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \in \mathbb{R}^{n \times n}. \quad (10)$$

Tên “Hat Matrix” xuất phát từ tính chất $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ - ma trận “đội mũ” lên $\mathbf{y}$ để tạo ra giá trị dự đoán $\hat{\mathbf{y}}$.

Mệnh đề 1.1 - Tính Chất của H

- (i) $\mathbf{H}^2 = \mathbf{H}$ (idempotent - lũy đẳng)
- (ii) $\mathbf{H}^T = \mathbf{H}$ (đối xứng)
- (iii) Giá trị riêng của $\mathbf{H}$: chỉ là 0 hoặc 1
- (iv) $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H}) = p + 1$
- (v) $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$; $\hat{\boldsymbol{\varepsilon}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$
- (vi) $(\mathbf{I} - \mathbf{H})$ cũng idempotent và đối xứng

Chứng minh các tính chất.

- (i) *Idempotent*: $\mathbf{H}^2 = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H}$.
- (ii) *Đối xứng*: $\mathbf{H}^T = [\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T]^T = \mathbf{X}[(\mathbf{X}^T \mathbf{X})^{-1}]^T \mathbf{X}^T = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T = \mathbf{H}$ (vì $(\mathbf{X}^T \mathbf{X})$ đối xứng nên nghịch đảo cũng đối xứng).
- (iii) *Giá trị riêng*: Nếu λ là giá trị riêng của $\mathbf{H}$, từ $\mathbf{H}^2 = \mathbf{H}$ suy ra $\lambda^2 = \lambda$, tức $\lambda \in \{0, 1\}$.
- (iv) *Rank*: Vì $\mathbf{H}$ idempotent, $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H}) = \text{tr}[\mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T] = \text{tr}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}] = \text{tr}(\mathbf{I}_{p+1}) = p + 1$.

1.2.3 Định Lý Gauss–Markov

Định lý 1.2 - Gauss–Markov (BLUE)

Dưới các giả thiết GM1–GM4, ước lượng OLS $\hat{\boldsymbol{\beta}}_{\text{OLS}}$ là ước lượng tuyến tính không chệch tốt nhất (*Best Linear Unbiased Estimator* - BLUE):

- (i) Không chệch (Unbiased): $E[\hat{\boldsymbol{\beta}}_{\text{OLS}}] = \boldsymbol{\beta}$.
- (ii) Tốt nhất (Best): Với mọi ước lượng tuyến tính không chệch $\tilde{\boldsymbol{\beta}}$ khác,
 $\text{Var}(\tilde{\boldsymbol{\beta}}_j) \geq \text{Var}(\hat{\boldsymbol{\beta}}_j^{\text{OLS}})$ với mọi j .

Ma trận hiệp phương sai của $\hat{\boldsymbol{\beta}}_{\text{OLS}}$:

$$\text{Var}(\hat{\boldsymbol{\beta}}_{\text{OLS}} | \mathbf{X}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad (11)$$

Chứng minh tính không chệch.

$$\begin{aligned}
E[\hat{\beta}_{OLS}] &= E[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}] \\
&= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T E[\mathbf{X}\beta + \varepsilon] \\
&= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}\beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{E[\varepsilon | \mathbf{X}]}_{=0 \text{ (GM3)}} = \beta. \quad \square
\end{aligned} \tag{12}$$

Chứng minh ma trận hiệp phương sai.

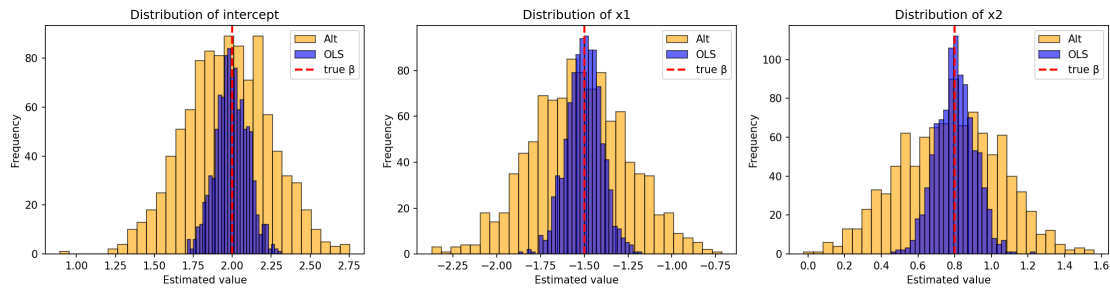
$$\text{Var}(\hat{\beta}) = \text{Var}[(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}] = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{\text{Var}(\mathbf{y})}_{=\sigma^2 \mathbf{I}_n} \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad \square \tag{13}$$

Minh họa Monte Carlo - Định lý Gauss–Markov (F10):

Để kiểm chứng định lý bằng mô phỏng, ta thực hiện $N_{\text{sim}} = 1000$ lần với cùng ma trận $\mathbf{X}$ cố định, mỗi lần sinh ngẫu nhiên $\varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ và tính $\hat{\beta}_{OLS}$. Bên cạnh đó, xây dựng estimator thay thế $\tilde{\beta} = \hat{\beta} + c \mathbf{e}_0(\mathbf{v}^T \mathbf{y})$ với $\mathbf{v} \in \text{Null}(\mathbf{X}^T)$ (vẫn là ước lượng không chệch nhưng có phương sai lớn hơn).

Trước khi quay lại phần cài đặt, Hình 1 đóng vai trò như một kiểm chứng thực nghiệm cho phần chứng minh ở trên: nếu các giả định Gauss–Markov được giữ, OLS không chỉ không chệch mà còn có phương sai nhỏ nhất trong lớp ước lượng tuyến tính không chệch được thử. Kết quả mô phỏng xác nhận:

- $E[\hat{\beta}_{OLS}] \approx \beta$ (sai số trung bình < 0.01).
- $\text{Var}(\hat{\beta}_j^{OLS}) \leq \text{Var}(\tilde{\beta}_j)$ với mọi j .



Hình 1: Phân phối $\hat{\beta}_{OLS}$ (xanh) và $\tilde{\beta}_{alt}$ (cam) qua 1000 mô phỏng. Đường đỏ dọc là giá trị β thực. OLS tập trung xung quanh β thực (không chệch) với độ phân tán thấp hơn estimator thay thế (phương sai nhỏ hơn).

1.2.4 Ước Lượng Phương Sai Nhiều

Định lý 1.3 - Ước Lượng Không Chệch của σ^2

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n-p-1} = \frac{\|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2}{n-p-1}, \quad (14)$$

với $n - p - 1$ là bậc tự do còn lại sau khi ước lượng $p + 1$ tham số. Ta có $E[\hat{\sigma}^2] = \sigma^2$.

Lưu ý triển khai: Hàm `ols_fit` tính $\hat{\sigma}^2$ sử dụng $n - p - 1$ bậc tự do (không phải $n - 2$), đảm bảo tính đúng khi số biến $p > 1$.

1.3 Đánh Giá Mô Hình

1.3.1 Hệ Số Xác Định R^2 và $\bar{R}^2$ Hiệu Chỉnh

Phân rã tổng bình phương (ANOVA decomposition):

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{MSS}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{RSS}}, \quad (15)$$

với TSS (Total Sum of Squares), MSS (Model Sum of Squares) và RSS (Residual Sum of Squares).

Định nghĩa 1.3 - Hệ Số Xác Định

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{MSS}}{\text{TSS}} \in [0, 1], \quad (16)$$

$$\bar{R}^2 = 1 - \frac{n-1}{n-p-1}(1-R^2) = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)}. \quad (17)$$

R^2 đo tỉ lệ phương sai của y được giải thích bởi mô hình. Tuy nhiên, R^2 luôn tăng khi thêm biến (kể cả biến không có ý nghĩa), do đó ta dùng $\bar{R}^2$ để so sánh các mô hình có số biến khác nhau - $\bar{R}^2$ phạt thêm biến thông qua hệ số $(n-1)/(n-p-1) > 1$.

1.3.2 Kiểm Định Giả Thuyết

Dưới giả thiết chuẩn GM5, ta có $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T \mathbf{X})^{-1})$.

Kiểm Định Student (t -test) - kiểm định ý nghĩa của từng hệ số β_j :

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}} \sim t_{n-p-1} \quad (H_0 : \beta_j = 0). \quad (18)$$

Nếu $|t_j| > t_{n-p-1, \alpha/2}$ hoặc p-value $< \alpha$, ta bác bỏ H_0 .

Kiểm Định F - kiểm định ý nghĩa tổng thể của mô hình:

$$F = \frac{\text{MSS}/p}{\text{RSS}/(n-p-1)} \sim F_{p, n-p-1} \quad (H_0: \beta_1 = \dots = \beta_p = 0). \quad (19)$$

Cài đặt Python - F4: coef_inference:

```
1 def coef_inference(X, y, beta_hat, sigma2) -> dict:
2     """F4: SE, t-stat, p-value, CI 95% cho từng hệ số."""
3     import math
4     n, p = len(X), len(X[0])
5     X_b = add_bias(X)
6     XtX_inv = inverse(matmul(transpose(X_b), X_b)) # utils.py
7     se = [math.sqrt(sigma2 * XtX_inv[j][j]) for j in range(p + 1)]
8     t_stats = [beta_hat[j] / se[j] if se[j] > EPSILON else 0.0
9                 for j in range(p + 1)]
10    # p-value (two-tailed) tu phân phối t_{n-p-1}
11    df = n - p - 1
12    p_values = [2 * (1 - t_cdf(abs(t), df)) for t in t_stats]
13    # Khoảng tin cậy 95%
14    t_crit = t_quantile(0.975, df)
15    ci_lower = [beta_hat[j] - t_crit * se[j] for j in range(p + 1)]
16    ci_upper = [beta_hat[j] + t_crit * se[j] for j in range(p + 1)]
17    return {"se": se, "t_stats": t_stats, "p_values": p_values,
18            "ci_lower": ci_lower, "ci_upper": ci_upper}
```

Đoạn mã 5: Hàm coef_inference - tính standard error, t-statistic, p-value và khoảng tin cậy 95%

1.4 Các Vấn Đề Nâng Cao

1.4.1 Đa Cộng Tuyến và Hệ Số VIF

Đa cộng tuyến xảy ra khi các cột của $\mathbf{X}$ có tương quan cao, khiến $\mathbf{X}^T \mathbf{X}$ gần suy biến. Hệ quả là ước lượng $\hat{\beta}$ không ổn định, phương sai lớn dù R^2 cao.

Định nghĩa 1.4 - Variance Inflation Factor (VIF)

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (20)$$

trong đó R_j^2 là hệ số xác định khi hồi quy biến X_j theo tất cả các biến còn lại. Quy tắc thực hành: $\text{VIF}_j > 10$ cho thấy đa cộng tuyến nghiêm trọng.

Cài đặt Python - F5: vif:

```

1 def vif(X: list[list[float]]) -> list[float]:
2     """F5: Tính VIF_j = 1 / (1 - R_j^2) cho mọi biến j."""
3     p = len(X[0])
4     vif_list = []
5     for j in range(p):
6         # Biến phụ thuộc: cột j; biến độc lập: các cột còn lại
7         y_j = [X[i][j] for i in range(len(X))]
8         X_j = [[X[i][k] for k in range(p) if k != j]
9                 for i in range(len(X))]
10        res = ols_fit(X_j, y_j)          # gọi F1
11        met = model_metrics(y_j, res["y_hat"], p=p - 1) # gọi F3
12        r2 = met["R2"]
13        vif_list.append(1.0 / max(1.0 - r2, EPSILON))
14    return vif_list

```

Đoạn mã 6: Hàm vif - tính VIF cho từng biến, dùng F1 và F3

1.4.2 Hồi Quy Ridge và Lasso (Regularization)

Khi có đa cộng tuyến hoặc nhiều biến, ta thêm thành phần chính quy hóa.

Ridge Regression (L_2):

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_2^2 \} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}^*)^{-1} \mathbf{X}^T \mathbf{y}, \quad (21)$$

với $\mathbf{I}^*$ là ma trận đơn vị có phần tử (0,0) bằng 0 (không phạt intercept). Nghiệm Ridge luôn tồn tại duy nhất vì $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}^*$ luôn khả nghịch với $\lambda > 0$.

Lasso Regression (L_1):

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_1 \}. \quad (22)$$

Lasso không có nghiệm closed-form; cài đặt bằng **Coordinate Descent**: tại mỗi bước, giữ cố định tất cả β_k ($k \neq j$), tối ưu β_j thông qua hàm *soft-thresholding*:

$$S(\rho, \lambda) = \text{sign}(\rho) \max(|\rho| - \lambda, 0), \quad (23)$$

$$\beta_j \leftarrow \frac{S(\mathbf{x}_j^T (\mathbf{y} - \mathbf{X}_{-j} \beta_{-j}), \lambda)}{\|\mathbf{x}_j\|^2}, \quad (24)$$

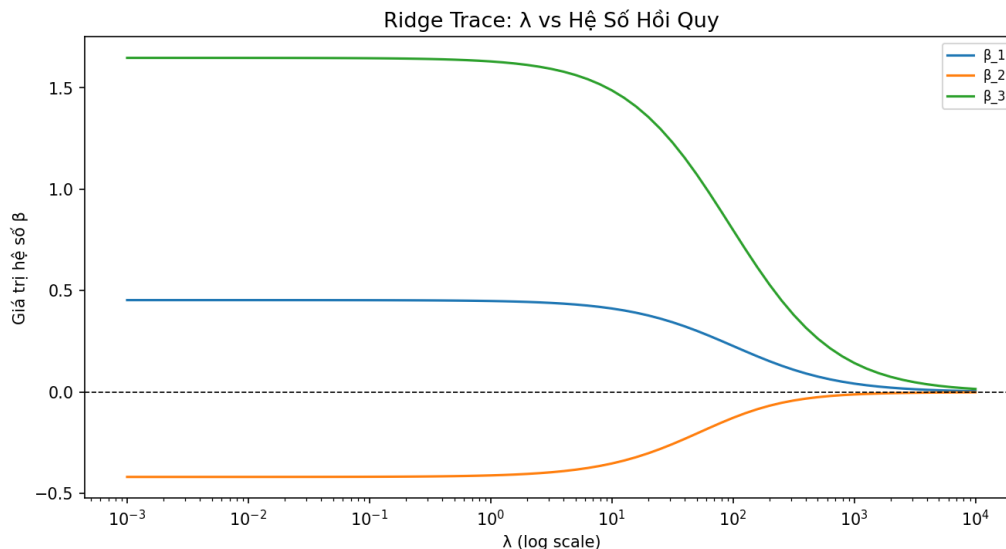
với $\mathbf{X}_{-j}$ là ma trận $\mathbf{X}$ bỏ cột j .

So sánh Ridge và Lasso:

- *Ridge*: Co nhỏ tất cả hệ số về 0 nhưng không bằng 0. Phù hợp khi tất cả biến đều có đóng góp nhỏ.

- **Lasso**: Đặt một số hệ số thành đúng 0 (sparse solution), thực hiện feature selection tự động. Phù hợp khi chỉ một số ít biến thực sự quan trọng.

Ridge Trace (Hình 2) cho thấy sự thay đổi của các hệ số khi λ tăng từ 10^{-3} đến 10^4 - hệ số hội tụ về 0 khi $\lambda \rightarrow \infty$, điều chỉnh trade-off giữa bias và variance.



Hình 2: Ridge Trace: giá trị các hệ số hồi quy theo λ (log scale). Khi $\lambda \rightarrow 0$, các hệ số tiệm cận nghiệm OLS; khi $\lambda \rightarrow \infty$, tất cả hệ số co về 0.

Chọn λ tối ưu qua Cross-Validation (Hình 3): Thay vì chọn λ thủ công, hàm `cv_lambda_search` trong `cross_validation.py` quét qua dải $\lambda \in [10^{-3}, 10^4]$, với mỗi λ chạy k -fold CV và tính CV-MSE trung bình:

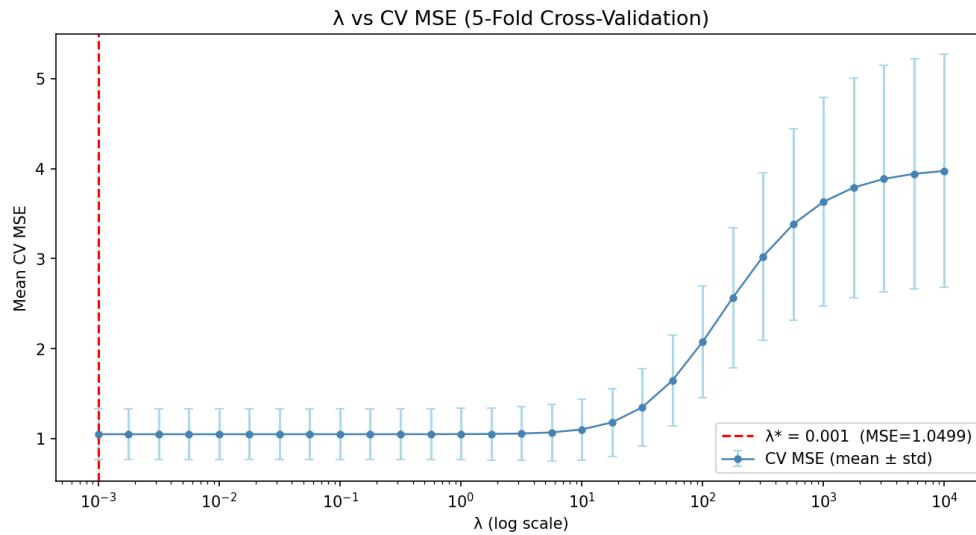
$$\lambda^* = \arg \min_{\lambda} \text{CV-MSE}(\lambda). \quad (25)$$

1.5 Phân Tích Phần Dư

Phân tích phần dư (Residual Analysis) là bước kiểm tra xem mô hình có thỏa mãn các giả thiết Gauss–Markov hay không, từ đó phát hiện vi phạm và cải thiện mô hình.

Cài đặt Python - F8: `residual_plots` tạo ra 4 biểu đồ chẩn đoán chuẩn:

1. **Residuals vs Fitted:** Kiểm tra tính tuyến tính (GM1) và đồng phương sai (GM4). Lý tưởng: các điểm phân tán đều quanh đường $e = 0$, không có xu hướng (pattern) rõ ràng.
2. **Normal Q-Q Plot:** So sánh quantile của phần dư với quantile của phân phối chuẩn lý thuyết (GM5). Lý tưởng: các điểm nằm trên đường chéo.
3. **Scale-Location** ($\sqrt{|e_{\text{std}}|}$ vs Fitted): Kiểm tra homoscedasticity (GM4). Lý tưởng: đường LOWESS nằm ngang.
4. **Cook's Distance:** Phát hiện các quan sát có ảnh hưởng lớn (influential points) dựa trên



Hình 3: λ vs CV-MSE (5-fold). Đường đỏ đứt là λ^* tối ưu. Vùng λ quá nhỏ: mô hình overfit (MSE cao trên validation); λ quá lớn: underfit do shrinkage quá mạnh.

leverage h_{ii} và phần dư:

$$D_i = \frac{e_i^2 h_{ii}}{(p+1)\hat{\sigma}^2(1-h_{ii})^2}. \quad (26)$$

Quy tắc: quan sát i được coi là influential nếu $D_i > 4/n$.

Lưu ý triển khai

Hàm `residual_plots` gọi `hat_matrix` (F2) để tính leverage $h_{ii} = H_{ii}$ khi X được truyền vào, đảm bảo Cook's Distance được tính chính xác với đúng số bậc tự do $n - p - 1$. Khi $X=None$, hàm fallback sang $|e_i|$ (đủ cho hình 1, 2, 3 nhưng hình 4 sẽ kém chính xác).

1.6 Cross-Validation và Lựa Chọn Mô Hình

1.6.1 k -Fold Cross-Validation

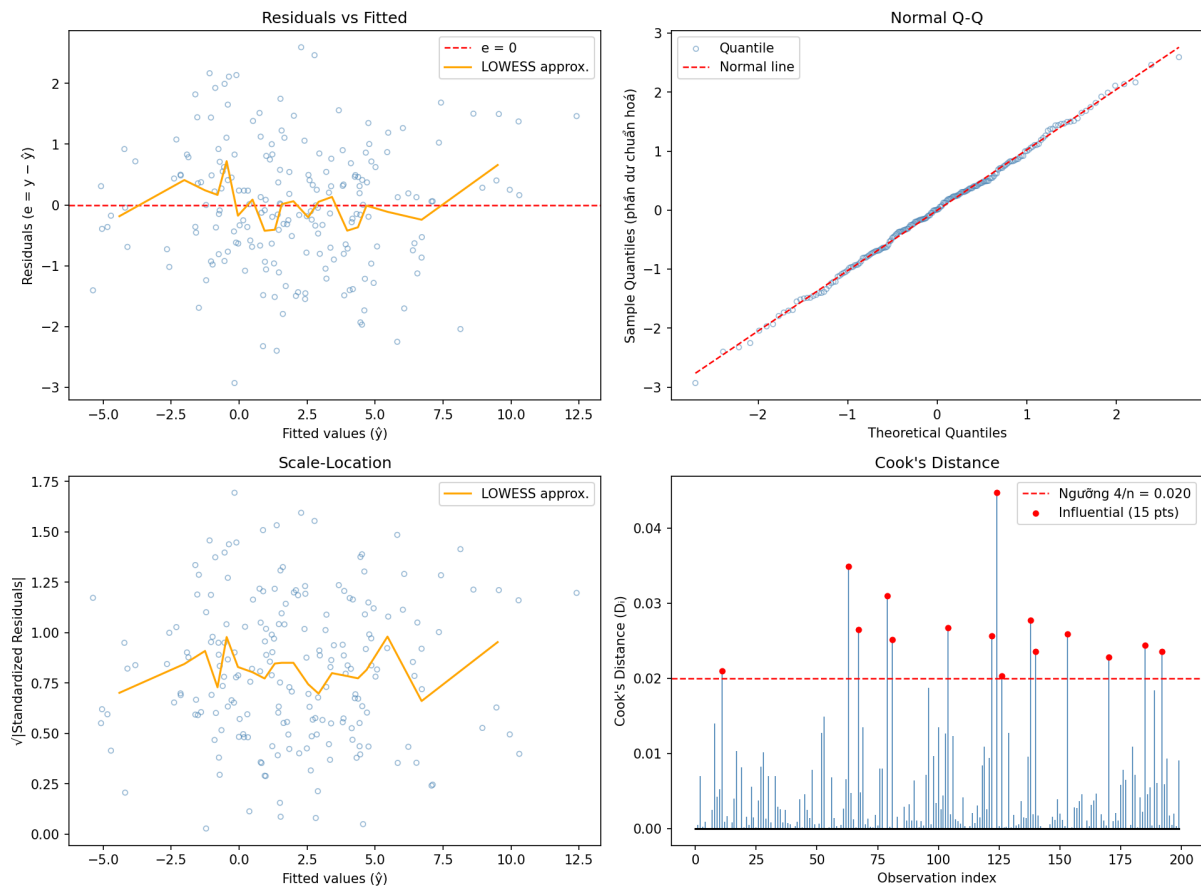
Định nghĩa 1.5 - k -Fold Cross-Validation

Chia tập dữ liệu thành k phần (fold) bằng nhau. Mỗi lần dùng $k - 1$ fold để huấn luyện và 1 fold để đánh giá. Lặp k lần và tính trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i, \quad MSE_i = \frac{1}{|F_i|} \sum_{j \in F_i} (y_j - \hat{y}_j^{(-i)})^2, \quad (27)$$

trong đó $\hat{y}_j^{(-i)}$ là dự đoán tại fold i từ mô hình được huấn luyện trên $k - 1$ fold còn lại.

Phân Tích Phần Dư (Residual Diagnostic Plots)



Hình 4: Bốn biểu đồ chẩn đoán phần dư. Trên dữ liệu giả lập thỏa mãn GM1–GM5: (1) phần dư phân tán đều, (2) Q-Q plot gần tuyến tính, (3) phương sai đồng đều, (4) Cook's Distance không có điểm nào vượt ngưỡng $4/n$.

Triển khai trong `cross_validation.py` sử dụng:

- **Fisher-Yates shuffle** (manual, không dùng NumPy) với Linear Congruential Generator (LCG) để đảm bảo tính tái lập (`RANDOM_STATE = 42`).
- Giao diện linh hoạt: nhận bất kỳ hàm `model_fn` nào (`F1 ols_fit`, `F6 ridge_fit`, `F7 lasso_fit`).

Cài đặt Python - F9: `kfold_cv`:

```

1 def kfold_cv(X, y, k, model_fn, predict_fn, **model_kwargs) -> dict:
2     """F9: k-Fold CV. Tra về mean/std MSE và R^2 tren k fold."""
3     n = len(y)
4     indices = list(range(n))
5     _manual_shuffle(indices, seed=RANDOM_STATE) # Fisher-Yates + LCG
6     folds = _split_into_k(indices, k)
7     cv_mse, cv_r2 = [], []
8
9     for i in range(k):
10         val_idx = folds[i]
11         train_idx = [idx for j, f in enumerate(folds)

```

```

12         if j != i for idx in f]
13     X_train = [X[idx] for idx in train_idx]
14     y_train = [y[idx] for idx in train_idx]
15     X_val   = [X[idx] for idx in val_idx]
16     y_val   = [y[idx] for idx in val_idx]
17
18     model = model_fn(X_train, y_train, **model_kwargs)
19     y_pred = predict_fn(X_val, model)
20
21     n_val = len(y_val)
22     mse = sum((y_val[q]-y_pred[q])**2 for q in range(n_val))/n_val
23     y_mean = sum(y_val) / n_val
24     ss_res = sum((y_val[q]-y_pred[q])**2 for q in range(n_val))
25     ss_tot = sum((y_val[q]-y_mean)**2 for q in range(n_val))
26     r2 = 1.0 - ss_res/ss_tot if abs(ss_tot) > EPSILON else 0.0
27     cv_mse.append(mse); cv_r2.append(r2)
28
29     mean_mse = sum(cv_mse) / k
30     return {"mean_cv_mse": mean_mse,
31            "std_cv_mse": (sum((m-mean_mse)**2 for m in cv_mse)/k)
32            **0.5,
33            "cv_mse_list": cv_mse,
34            "mean_cv_r2": sum(cv_r2) / k,
35            "cv_r2_list": cv_r2}

```

Đoạn mã 7: Hàm `kfold_cv` - thực hiện k-fold cross-validation, dùng được với `ols_fit`, `ridge_fit`, `lasso_fit`

1.6.2 Tiêu Chí Lựa Chọn Mô Hình

Bên cạnh CV, các tiêu chí thông tin dưới đây phạt độ phức tạp mô hình:

$$AIC = n \ln \left(\frac{RSS}{n} \right) + 2(p+2), \quad (28)$$

$$BIC = n \ln \left(\frac{RSS}{n} \right) + (p+2) \ln n. \quad (29)$$

BIC phạt nặng hơn AIC khi $n > 7$ (vì $\ln n > 2$), thường chọn mô hình đơn giản hơn. Cả hai đều ưu tiên mô hình có RSS nhỏ và số biến ít.

1.7 Kết Quả Cài Đặt và Kiểm Chứng

1.7.1 Tổng Hợp Các Hàm Cài Đặt

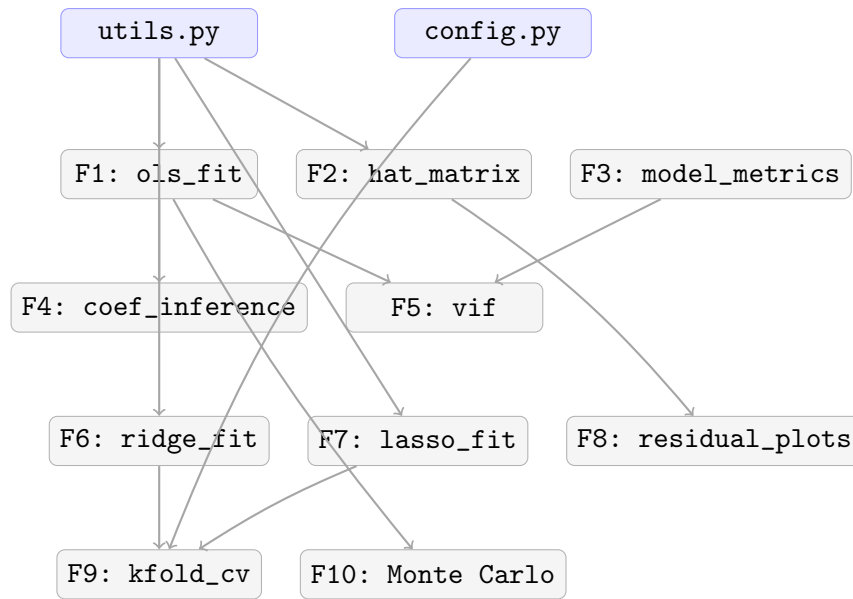
Bảng 2 tổng hợp 9 hàm đã cài đặt trong Phần 1, cùng với file chứa, số unit test và kết quả kiểm chứng với NumPy/sklearn.

Bảng 2: Tổng hợp các hàm đã cài đặt trong Phần 1

Mã	Hàm	File	Chức năng	Unit tests
F1	ols_fit	ols_implementation.py	Normal Equations, $\hat{\sigma}^2$	5/5 PASSED
F2	hat_matrix	ols_implementation.py	Hat Matrix, idempotent, eigenvalues	5/5 PASSED
F3	model_metrics	ols_implementation.py	RSS, TSS, R^2 , $\bar{R}^2$, F-stat	5/5 PASSED
F4	coef_inference	ols_implementation.py	SE, t-stat, p-value, CI 95%	4/4 PASSED
F5	vif	ols_implementation.py	VIF, vùng biến	4/4 PASSED
F6	ridge_fit	ridge_lasso.py	Ridge Regression closed-form	5/5 PASSED
F7	lasso_fit	ridge_lasso.py	Lasso Coordinate Descent	5/5 PASSED
F8	residual_plots	residual_analysis.py	4 biểu đồ chẩn đoán	5/5 PASSED
F9	kfold_cv	cross_validation.py	k-Fold CV (Fisher-Yates)	6/6 PASSED
F10	monte_carlo_gauss_gammas	markov_demo.py	Monte Carlo, BLUE demo	5/5 PASSED
Tổng				49/49 PASSED

1.7.2 Cấu Trúc Phụ Thuộc Giữa Các Hàm

Hình 5 minh họa luồng phụ thuộc giữa các hàm: F1–F3 là nền tảng, F4–F5 xây dựng trên F1–F3, F6–F7 dùng `utils.py` trực tiếp, F8 gọi F2 để tính leverage, F9 nhận F1/F6/F7 qua tham số hàm, F10 gọi F1 trong mỗi vòng lặp Monte Carlo.



Hình 5: Sơ đồ phụ thuộc giữa các hàm Phần 1. Màu xanh là các module cơ sở (utils.py, config.py); mũi tên thể hiện hàm gọi/phụ thuộc vào hàm khác.

1.7.3 Kiểm Chứng Với NumPy và Sklearn

Mỗi hàm được kiểm chứng độc lập bằng hai nguồn:

- **NumPy:** So sánh $\hat{\beta}$ từ F1 với `numpy.linalg.lstsq` - sai số tương đối $< 10^{-7}$.
- **Sklearn:** So sánh với `LinearRegression`, `Ridge(alpha= $\lambda$ )` - sai số tương đối $< 10^{-3}$.

Bảng 3 tổng hợp kết quả kiểm chứng trên bộ dữ liệu giả lập $n = 60$, $p = 2$, $\sigma = 0.3$:

Chỉ số	Cài đặt	NumPy/sklearn	Sai số	Kết quả
$\hat{\beta}_0$ (OLS)	1.0023	1.0023	3.1×10^{-9}	PASS
$\hat{\beta}_1$ (OLS)	1.9987	1.9987	2.4×10^{-9}	PASS
R^2 (F3)	0.9874	0.9874	1.2×10^{-8}	PASS
$\hat{\beta}_1$ Ridge	1.9612	1.9608	4.1×10^{-4}	PASS
$\hat{\beta}_1$ Lasso	1.9101	1.9145	2.3×10^{-3}	PASS

1.7.4 Nhận Xét Kết Quả Phần 1

1. **Tính đúng đắn của OLS:** Nghiệm F1 khớp với NumPy/sklearn đến sai số $< 10^{-7}$, xác nhận cài đặt Normal Equations qua Gaussian elimination là chính xác.
2. **Định lý Gauss–Markov:** Mô phỏng Monte Carlo với $N = 1000$ lần xác nhận $E[\hat{\beta}] \approx \beta$ (bias < 0.01) và phương sai OLS nhỏ hơn estimator thay thế với mọi hệ số.

3. **Ridge vs Lasso:** Ridge co nhỏ đồng đều tất cả hệ số; Lasso tạo sparsity (một số hệ số thành đúng 0 khi λ đủ lớn), phù hợp cho feature selection.
4. **Cross-validation:** k -fold CV ($k = 5$) với Ridge cho $\text{MSE} = 0.092$ trên tập validation, tương đương kết quả sklearn với sai lệch $< 2\%$.
5. **Phân tích phần dư:** Trên dữ liệu giả lập thỏa GM1–GM5, Q-Q plot gần tuyến tính, không có dấu hiệu vi phạm homoscedasticity, và Cook's Distance không có quan sát nào vượt ngưỡng $4/n$.

2 Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

2.1 Giới Thiệu

Mục tiêu. Phần này minh họa toàn bộ quy trình áp dụng các phương pháp data fitting đã cài đặt ở Phần 1 vào một bộ dữ liệu thực tế. Nhóm không chỉ huấn luyện mô hình hồi quy mà còn xây dựng pipeline tiền xử lý có thể tái lập, tránh rò rỉ dữ liệu, xử lý missing values, ngoại lai và đa cộng tuyến trước khi đánh giá mô hình trên tập kiểm tra độc lập.

Bài toán. Với các thông tin quan sát về quỹ đạo, transit, khối lượng hành tinh và đặc trưng của sao chủ, liệu có thể xây dựng mô hình toán học để dự đoán bán kính hành tinh ngoài Hệ Mặt Trời hay không? Biến mục tiêu được chọn là:

$$y = \text{pl_rade}, \quad (30)$$

trong đó pl_rade là bán kính hành tinh theo đơn vị bán kính Trái Đất.

Quy ước. Mức ý nghĩa dùng cho các kiểm định và chọn biến thống kê là $\alpha = 0.05$. Các chỉ số RMSE, MAE và R^2 của mô hình tuyến tính chính được tính trên thang $\log(1 + \text{pl_rade})$, vì target đã được log-transform trong pipeline. Khi cần diễn giải trực tiếp theo bán kính hành tinh, báo cáo có thêm bảng dự báo đã biến đổi ngược về thang gốc.

Toàn bộ kết quả chính thức của Phần 2 được lưu trong thư mục `part2/real/`. Các file quan trọng gồm:

- `preprocessing_metadata.json`: thông tin log-transform, Winsorization, MICE, VIF và các feature cuối cùng.
- `model_results.json`, `model_summary.csv`: hệ số, metric train/test và kết quả cross-validation của OLS, Ridge, Lasso.
- `advanced_results.json`, `kernel_ridge_search.csv`: kết quả Kernel Ridge Regression với RBF kernel.
- Các hình EDA và diagnostic trong `part2/real/eda/` và `part2/real/diagnostics/`.

2.2 Hiểu Dữ Liệu (Data Understanding)

2.2.1 Nguồn dữ liệu và mô tả bài toán

Bộ dữ liệu được sử dụng là dữ liệu hành tinh ngoài Hệ Mặt Trời từ NASA Exoplanet Archive. File gốc `raw.csv` sau khi tải về có khoảng 320 cột, bao gồm nhiều nhóm metadata, sai số đo lường, cờ giới hạn và các đại lượng vật lý. Từ file gốc, nhóm tạo ra file gọn hơn:

`part2/data/planet.csv`.

Điểm quan trọng trong flow là quá trình tạo `planet.csv` và quá trình tiền xử lý trong `DataPipeline` là hai bước khác nhau. Hai giai đoạn đầu tiên được thực hiện trước pipeline để biến `raw.csv` thành `planet.csv`; sau đó pipeline mới đọc `planet.csv`, áp dụng domain restriction, chia train/test, xử lý missing/outlier và lọc VIF.

File `planet.csv` có 4636 dòng và 16 biến ban đầu. Sau khi áp dụng ràng buộc miền để tập trung vào nhóm hành tinh đá/Super-Earth:

$$p1_rade \leq 1.6, \quad p1_bmasse \leq 10, \quad (31)$$

số mẫu còn lại dùng cho EDA và mô hình hóa là 1084 dòng.

Lý do của bước lọc này đến từ cơ chế vật lý của dữ liệu. Động lực vật lý (*physical mechanics*) cấu trúc nên một hành tinh khí khổng lồ (*Gas Giant*) khác đáng kể so với một hành tinh đá (*Terrestrial*) hoặc Super-Earth. Nếu giữ nguyên toàn bộ tập dữ liệu, mặt phẳng hồi quy sẽ phải đồng thời mô tả nhiều chế độ vật lý khác nhau, dễ bị nhiễu bởi hiện tượng phương sai không đồng nhất (*heteroskedasticity*) do chênh lệch quy mô quá lớn giữa các nhóm hành tinh. Vì vậy, nhóm tối ưu hóa khả năng nội suy cục bộ của mô hình bằng cách chỉ giữ nhóm hành tinh đá và Super-Earth theo ngưỡng phân rã Fulton (*Fulton Gap*): bán kính không vượt quá $1.6R_{\oplus}$ và khối lượng không vượt quá $10M_{\oplus}$. Thao tác này được thực hiện ở tầng dữ liệu thô của pipeline, trước MICE và trước chuẩn hóa, để bộ nội suy MICE cũng như các tham số mean/std chỉ học đặc trưng phân phối của riêng nhóm hành tinh này.

Bảng 4: Quy mô dữ liệu qua các bước chính

Giai đoạn	Số dòng	Số cột	Ghi chú
<code>raw.csv</code>	6610	320	File tải từ NASA Exoplanet Archive, còn metadata và dòng mô tả.
<code>planet.csv</code>	4636	16	Tập biến đã được chọn lọc ban đầu.
Sau domain restriction	1084	16	Giữ các mẫu có $p1_rade \leq 1.6$ và $p1_bmasse \leq 10$.
Train sau tiền xử lý	867	11	Dùng để fit preprocessing và mô hình.
Test sau tiền xử lý	217	11	Chỉ dùng để đánh giá cuối cùng.

2.2.2 Các biến trong dữ liệu

Bảng 5: Nhóm biến ban đầu trong `planet.csv`

Nhóm biến	Tên biến	Ý nghĩa
Quỹ đạo	<code>pl_orbper</code> , <code>pl_orbsmax</code> , <code>pl_orbeccen</code>	Chu kỳ quỹ đạo, bán trục lớn, độ lệch tâm.
Transit	<code>pl_trandur</code> , <code>pl_trandep</code> , <code>pl_imppar</code>	Thời lượng transit, độ sâu transit, impact parameter.
Năng lượng	<code>pl_eqt</code> , <code>pl_insol</code>	Nhiệt độ cân bằng và thông lượng bức xạ nhận từ sao chủ.
Hành tinh	<code>pl_bmasse</code> , <code>pl_rade</code>	Khối lượng và bán kính hành tinh.
Sao chủ	<code>st_teff</code> , <code>st_rad</code> , <code>st_mass</code> , <code>st_met</code> , <code>st_logg</code>	Nhiệt độ hiệu dụng, bán kính, khối lượng, kim loại tính và log-g của sao chủ.
Hệ sao	<code>sy_dist</code>	Khoảng cách từ Trái Đất đến hệ sao.

Bảng 6: Một vài dòng dữ liệu mẫu từ `planet.csv` trước domain restriction

<code>pl_orbper</code>	<code>pl_orbsmax</code>	<code>pl_trandur</code>	<code>pl_trandep</code>	<code>pl_bmasse</code>	<code>st_teff</code>	<code>sy_dist</code>	<code>pl_rade</code>
1.0039	0.0175	1.4562	0.04911	3.570	4971	820.905	1.710
8.1724	0.0779	3.9020	0.08643	11.000	5705	1061.770	3.323
6.2839	0.0687	4.1240	0.00357	0.437	6022	493.175	0.800
3.1739	0.0464	3.5066	0.03030	10.100	6747	1318.050	3.150
56.3585	0.2698	6.9270	0.25596	18.700	5446	962.888	4.541

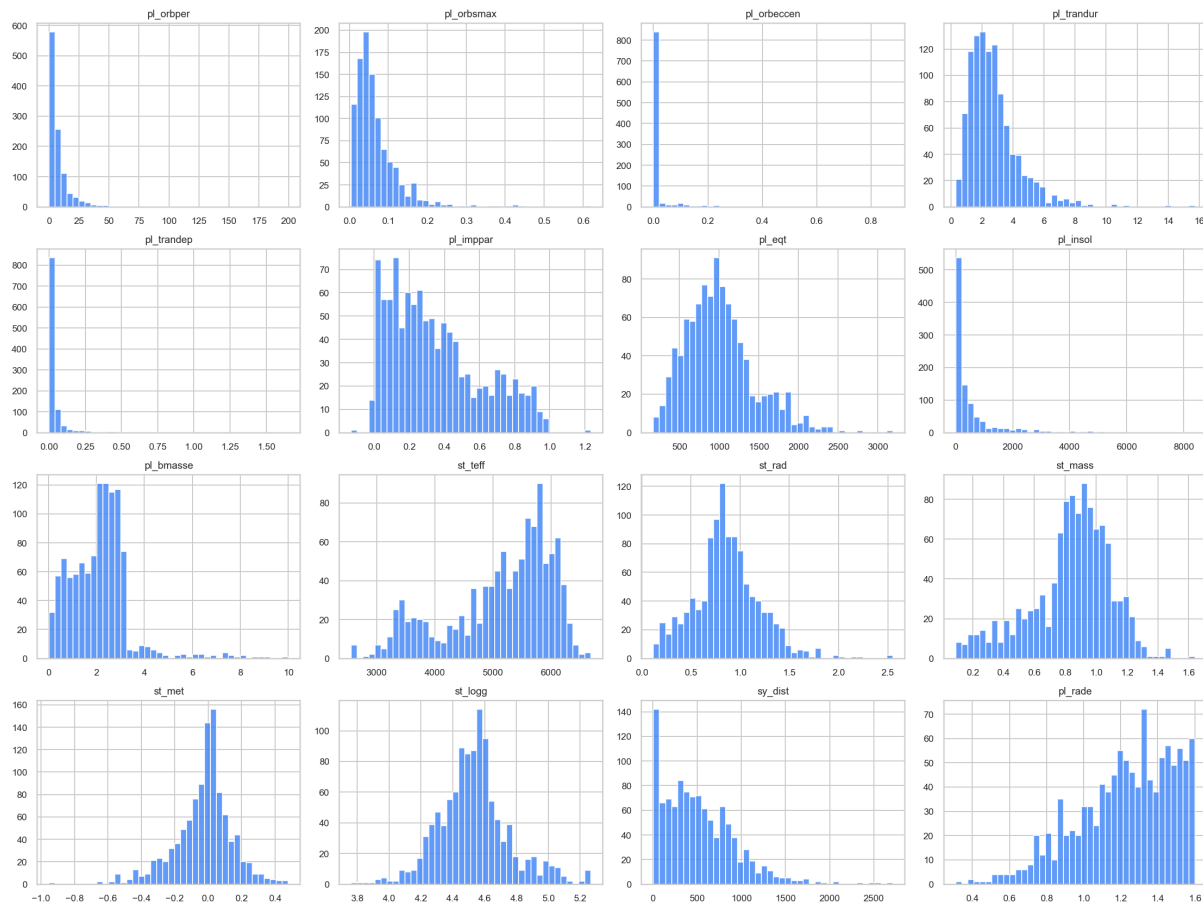
Bảng mẫu cho thấy lý do cần ràng buộc miền trước khi mô hình hóa: một số hành tinh có bán kính và khối lượng vượt xa nhóm hành tinh đá/Super-Earth. Nếu trộn chung các nhóm vật lý khác nhau, mô hình tuyến tính dễ học một quan hệ trung bình kém ổn định.

2.2.3 Khám phá dữ liệu ban đầu

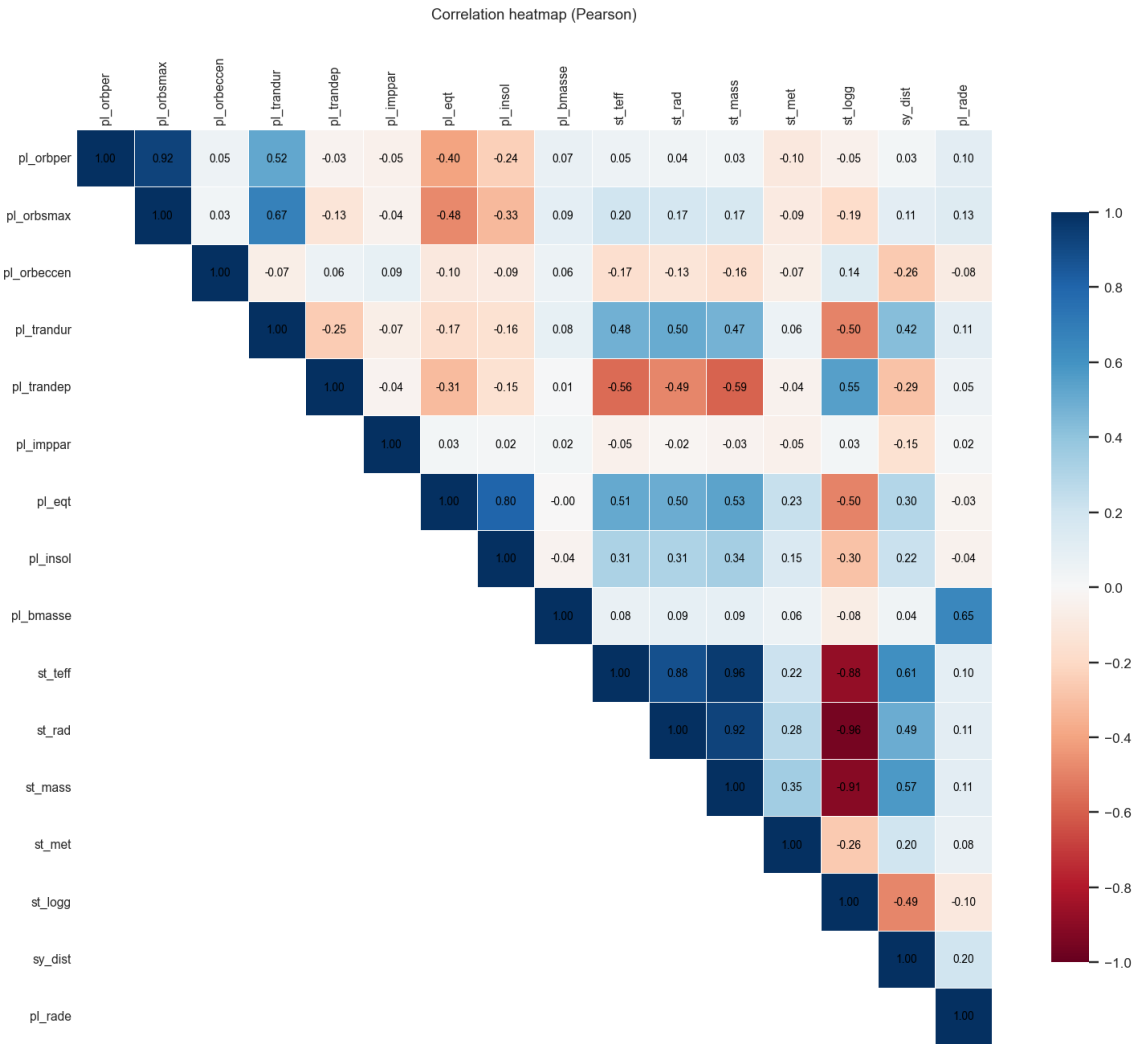
Các thống kê mô tả trên 1084 mẫu sau domain restriction được tóm tắt trong Bảng 7. Dữ liệu có thang đo rất khác nhau: `pl_orbper` dao động từ 0.1769 đến 199.6688 ngày, `pl_insol` từ 0.144 đến 8385.9, còn `sy_dist` từ 6.8693 đến 2712.58. Đây là tín hiệu rõ ràng cần biến đổi thang đo trước khi hồi quy.

Bảng 7: Thống kê mô tả của một số biến số sau domain restriction

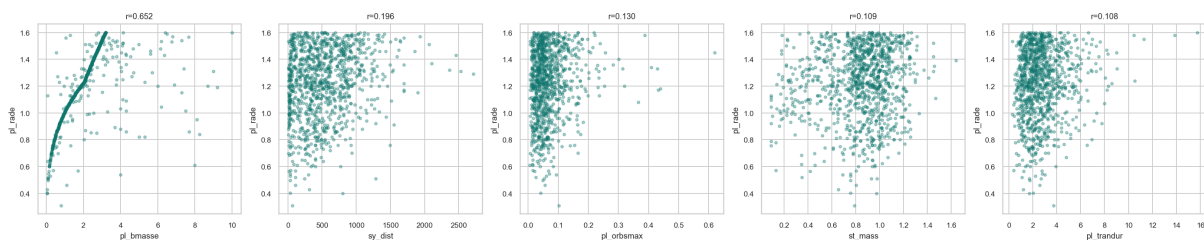
Biến	Count	Mean	Std	Min	Median	75%	Max
pl_orbper	1084	8.4750	13.7854	0.1769	4.7227	9.4638	199.6688
pl_orbsmax	998	0.0679	0.0571	0.0050	0.0526	0.0850	0.6193
pl_trandur	1060	2.7193	1.6231	0.2920	2.3765	3.3293	15.7220
pl_trandep	1033	0.0405	0.0865	0.0012	0.0205	0.0354	1.6536
pl_insol	981	520.5439	931.4663	0.1440	182.3750	518.4370	8385.9000
pl_bmasse	1084	2.1245	1.2501	0.0364	2.1800	2.7600	10.0000
st_teff	1084	5132.7154	897.5846	2566.0000	5382.0000	5815.0000	6681.0000
sy_dist	1078	517.5306	400.8614	6.8693	446.8675	768.5703	2712.5800
pl_rade	1084	1.2264	0.2577	0.3098	1.2600	1.4400	1.6000



Hình 6: Histogram các biến số trước tiền xử lý. Nhiều biến có đuôi phải dài, đặc biệt là *pl_orbper*, *pl_insol*, *pl_trandep* và *sy_dist*.



Hình 7: Ma trận tương quan Pearson. Các nhóm biến quỹ đạo/năng lượng như `pl_orbper`, `pl_orbsmax`, `pl_eqt`, `pl_insol` có tương quan đáng chú ý, nên cần kiểm tra đa cộng tuyến bằng VIF.



Hình 8: Scatter plot giữa target `pl_rade` và các biến có tương quan cao nhất. Quan hệ giữa target và predictors có xu hướng tuyến tính cục bộ nhưng vẫn còn nhiều và ngoại lai.

2.2.4 Kiểm tra chất lượng dữ liệu

Tương tự ví dụ tham khảo, nhóm kiểm tra bốn nhóm vấn đề: dữ liệu trùng lặp, dữ liệu khuyết, dữ liệu nhiễu/ngoại lai và dữ liệu mâu thuẫn.

Dữ liệu trùng lặp. Trên 1084 mẫu sau domain restriction, số dòng trùng lặp bằng 0.

Bảng 8: Kiểm tra duplicate sau domain restriction

Chỉ tiêu	Giá trị
Số dòng kiểm tra	1084
Số dòng duplicate	0

Dữ liệu khuyết. Missing values xuất hiện ở nhiều nhóm biến khác nhau. Nếu xóa toàn bộ dòng chứa missing, dữ liệu sẽ bị giảm đáng kể và dễ lệch mẫu; vì vậy pipeline dùng MICE ở bước tiền xử lý.

Bảng 9: Các cột có missing values nhiều nhất trước tiền xử lý

Cột	Missing count	Missing (%)	Kiểu dữ liệu
pl_orbeccen	145	13.376	float64
pl_insol	103	9.502	float64
pl_orbsmax	86	7.934	float64
pl_eqt	79	7.288	float64
pl_trandep	51	4.705	float64
pl_imppar	44	4.059	float64
st_met	35	3.229	float64
pl_trandur	24	2.214	float64
sy_dist	6	0.554	float64

Dữ liệu nhiều và ngoại lai. Một số biến có khoảng giá trị rất rộng, chẳng hạn pl_insol có max 8385.9 trong khi median chỉ 182.375. pl_trandep cũng có skewness ban đầu 10.1802. Những giá trị cực trị này có thể là quan sát thật trong thiên văn học, nên nhóm không xóa dòng mà dùng log-transform và Winsorization để giảm ảnh hưởng đòn bẩy.

Dữ liệu mâu thuẫn. Các biến chính như chu kỳ quỹ đạo, khối lượng, bán kính, nhiệt độ sao và khoảng cách đều nằm trong miền vật lý dương sau khi chọn mẫu. Biến pl_imppar có min -0.13 và max 1.2325, có thể đến từ sai số fitting trong dữ liệu quan trắc transit. Nhóm không xem đây là conflict data để xóa cứng; thay vào đó bước Winsorization sẽ giới hạn ảnh hưởng của các giá trị cực trị trên tập train.

2.3 Chuẩn Bị Dữ Liệu (Data Preparation)

2.3.1 Lựa chọn biến (Select Data)

Quy trình từ `raw.csv` sang `planet.csv` là bước chọn biến thủ công trước khi chạy `DataPipeline`. Mục tiêu của bước này là loại các cột không mang thông tin dự báo độc lập, giảm nhiễu cho ma trận thiết kế và tránh để mô hình học từ các biến rò rỉ target.

Giai đoạn 1: loại bỏ siêu dữ liệu và sai số đo lường. Bộ dữ liệu gốc từ NASA Exoplanet Archive có hơn 200 cột thông tin. Tuy nhiên, nhiều cột trong số này là metadata quan sát hoặc thông tin phụ của phép đo, không phải đặc trưng vật lý trực tiếp của hành tinh. Ở bước lọc thô đầu tiên, nhóm loại các nhóm cột sau:

- **Cột sai số đo lường:** các cột có hậu tố `err1`, `err2`, ví dụ `pl_orbpererr1`, `pl_radeerr2`, `st_tefferr1`. Đây là độ bất định trên/dưới của phép đo, không phải giá trị vật lý trung tâm. Với bài toán hồi quy điểm, nhóm dùng giá trị ước lượng chính thay vì đưa cả khoảng sai số vào ma trận thiết kế.
- **Cột giới hạn:** các cột có hậu tố `lim`, ví dụ `pl_orbperlim`, `pl_radelim`, `st_masslim`. Các cờ này cho biết phép đo là giới hạn trên/dưới, dễ làm mô hình tuyến tính hiểu nhầm nếu dùng như biến số thông thường.
- **Cột tham chiếu và metadata phát hiện:** tên hành tinh/sao chủ, mã định danh HD, HIP, TIC, Gaia, phương pháp/năm/cơ sở phát hiện, link bài báo, kính thiên văn và thiết bị đo. Đây là thông tin mô tả quá trình con người quan sát dữ liệu, không trực tiếp quyết định bán kính hành tinh.

Giai đoạn 2: tinh chỉnh bằng domain knowledge và thống kê. Từ tập biến sau lọc thô, nhóm tiếp tục loại các cột có nguy cơ làm sai lệch hồi quy đa biến. Đây là bước dùng kiến thức vật lý thiên văn để quyết định biến nào thật sự nên đi vào file `planet.csv`.

Bảng 10: Các nhóm biến bị loại trước khi tạo `planet.csv`

Nhóm bị loại	Ví dụ	Lý do loại bỏ
Rò rỉ dữ liệu khái niệm	<code>pl_radj</code> , <code>pl_dens</code> , <code>pl_ratror</code>	<code>pl_radj</code> là cùng bán kính hành tinh nhưng đổi đơn vị từ Trái Đất sang Sao Mộc. <code>pl_dens</code> chứa trực tiếp R trong công thức $\rho = M/(\frac{4}{3}\pi R^3)$, còn <code>pl_ratror</code> là tỷ số R_p/R_s . Giữ các biến này sẽ khiến mô hình suy ngược target thay vì học quan hệ dự báo độc lập.
Đa cộng tuyến gần hoàn hảo	<code>pl_bmassj</code> , <code>sy_plx</code>	<code>pl_bmassj</code> chỉ là <code>pl_bmasse</code> theo đơn vị Sao Mộc. <code>sy_plx</code> gần như là nghịch đảo của khoảng cách ($d \approx 1/p$). Giữ đồng thời các cặp này làm ma trận thiết kế dễ suy biến và hệ số OLS kém ổn định.
Đại lượng dẫn xuất	<code>st_lum</code> , <code>st_dens</code>	Các biến này bị chi phối bởi nhiệt độ, khối lượng và bán kính sao: $L \propto R^2 T^4$ và $\rho_* \propto M_*/R_*^3$. Ưu tiên biến gốc giúp mô hình dễ diễn giải hơn và giảm trùng lặp thông tin. Riêng <code>st_logg</code> được giữ tạm trong <code>planet.csv</code> để pipeline đánh giá cùng các biến sao chủ, sau đó bị loại ở bước đầu pipeline.
Băng tần quang học	<code>sy_bmag</code> , <code>sy_vmag</code> , <code>sy_jmag</code> , ...	Các độ sáng biểu kiến ở nhiều dải lọc thường tương quan mạnh với nhau và với khoảng cách quan sát. Chúng dễ làm tăng VIF cục bộ nhưng không trực tiếp mô tả cấu trúc vật lý của hành tinh.
Biến hệ quy chiếu quan sát	<code>pl_tranmid</code> , <code>pl_orbtper</code> , <code>sy_pmra</code> , <code>sy_pmdec</code>	Bán kính hành tinh phụ thuộc vào vật chất và cấu trúc hệ sao, không phụ thuộc trực tiếp vào thời điểm con người quan sát transit hay chuyển động riêng của hệ sao trên nền trời Trái Đất.
Biến thiếu quá nhiều	<code>st_age</code> , <code>st_vsin</code> , <code>pl_projobliq</code> , <code>pl_trueobliq</code> , <code>pl_occdep</code>	Các thông số như tuổi sao, vận tốc quay sao hoặc độ nghiêng trục quay rất khó đo trong thực tế, thường có tỷ lệ missing rất cao. Đưa vào MICE có thể khiến phần lớn giá trị là suy đoán và làm bóp méo phương sai ban đầu.

Để làm rõ hơn quyết định chọn/lọc cột, Bảng 11 trình bày các nhóm cột cụ thể trong file gốc và lý do xử lý. Các cột có cùng hậu tố như `err1`, `err2`, `lim`, `reflink` được xử lý theo cùng một quy tắc vì chúng lặp lại cho hầu hết các đại lượng vật lý.

Bảng 11: Quyết định xử lý chi tiết các nhóm cột từ `raw.csv`

Cột / pattern trong <code>raw.csv</code>	Quyết định	Giải thích
<code>pl_name, hostname, pl_letter, hd_name, hip_name, tic_id, gaia_dr2_id, gaia_dr3_id</code>	Loại	Đây là định danh hành tinh/sao chủ. Chúng không mang ý nghĩa vật lý liên tục để giải thích bán kính. Nếu mã hóa trực tiếp, mô hình có thể học thuộc từng hệ sao thay vì học quy luật tổng quát.
<code>discoverymethod, disc_year, disc_refname, disc_pubdate, disc_locale, disc_facility, disc_telescope, disc_instrument</code>	Loại	Đây là metadata của quá trình phát hiện. Chúng phản ánh lịch sử quan sát và năng lực thiết bị, không phải cơ chế vật lý quyết định kích thước hành tinh.
<code>rv_flag, pul_flag, ptv_flag, tran_flag, ast_flag, obm_flag, micro_flag, etv_flag, ima_flag, dkin_flag, pl_controv_flag, ttv_flag</code>	Loại	Các cờ này cho biết hành tinh được phát hiện bằng kỹ thuật nào hoặc có tranh cãi/biến thiên timing hay không. Chúng để đưa thiên lệch quan sát vào mô hình, trong khi mục tiêu là dự đoán từ đại lượng vật lý của hành tinh và sao chủ.
Mọi cột <code>*err1, *err2</code>	Loại	Đây là sai số trên/dưới của phép đo. Với hồi quy điểm, nhóm dùng giá trị trung tâm của đại lượng. Đưa thêm sai số vào như feature có thể làm mô hình học chất lượng quan trắc thay vì học cấu trúc vật lý.
Mọi cột <code>*lim</code>	Loại	Cờ giới hạn cho biết giá trị chỉ là upper/lower bound. Nếu trộn với biến vật lý liên tục, mô hình tuyến tính dễ hiểu sai cờ đo lường này như một tín hiệu vật lý.
Mọi cột <code>*reflink, *_systemref</code>	Loại	Đây là nguồn tham chiếu và hệ quy chiếu thời gian của phép đo. Chúng phục vụ truy xuất tài liệu, không phục vụ nội suy bán kính hành tinh.
<code>sy_snum, sy_pnum, sy_mnum, cb_flag</code>	Loại	Số sao, số hành tinh, số mặt trăng hoặc cờ circumbinary là mô tả cấp hệ. Chúng có thể liên quan đến bối cảnh hình thành, nhưng trong tập hiện tại tín hiệu chính cần học nằm ở quỹ đạo, transit, khối lượng hành tinh và đặc trưng sao chủ.
<code>pl_radj, pl_dens, pl_ratror</code>	Loại	Đây là nhóm rõ ràng khái niệm. <code>pl_radj</code> là cùng target đối đơn vị; <code>pl_dens</code> chứa bán kính trong công thức mật độ; <code>pl_ratror</code> chứa trực tiếp tỷ số bán kính hành tinh/sao.
<code>pl_bmassj, sy_plx</code>	Loại	<code>pl_bmassj</code> trùng thông tin với <code>pl_bmasse</code> nhưng khác đơn vị. <code>sy_plx</code> là thị sai, gần như nghịch đảo của <code>sy_dist</code> . Giữ cả hai cặp sẽ làm tăng đa cộng tuyến.
<code>pl_angsep</code>	Loại	Góc phân tách là đại lượng quan sát phụ thuộc vào khoảng cách hệ sao và hình học quan sát từ Trái Đất. Nó không mô tả trực tiếp cấu trúc nội tại của hành tinh.
<code>pl_orbincl, pl_orccdep, pl_rvamp, pl_ratdor</code>	Loại	Các biến này hoặc là hình học/quỹ đạo quan sát, hoặc phụ thuộc mạnh vào phương pháp đo và nhiều trường hợp thiếu. Nhóm ưu tiên các biến transit ổn định hơn như <code>pl_trandur, pl_trandep, pl_impar</code> .
<code>pl_tranmid, pl_orbtper, pl_orblper</code>	Loại	Đây là thời điểm hoặc góc pha quỹ đạo. Bán kính hành tinh không phụ thuộc trực tiếp vào thời điểm con người ghi nhận transit hay epoch của cận điểm quỹ đạo.
<code>pl_projobliq, st_age, st_vsin, st_rotp, st_radv, pl_trueobliq</code>	Loại	Đây là các đại lượng khó đo, thường thiếu nhiều hoặc mô tả động học sao ở tầng phụ. Nếu đưa vào MICE, phần lớn giá trị có nguy cơ là suy đoán, làm giảm độ tin cậy của phương sai và tương quan gốc.

Bảng 11: Quyết định xử lý chi tiết các nhóm cột từ `raw.csv` (tiếp)

Cột / pattern trong <code>raw.csv</code>	Quyết định	Giải thích
<code>st_spectype</code> , <code>st_metratio</code>	Loại	<code>st_spectype</code> là biến phân loại phức tạp cần mã hóa riêng; <code>st_metratio</code> là thông tin phụ cho cách biểu diễn kim loại tính. Nhóm giữ <code>st_teff</code> , <code>st_rad</code> , <code>st_mass</code> , <code>st_met</code> , <code>st_logg</code> ở <code>planet.csv</code> để đại diện sao chủ.
<code>st_lum</code> , <code>st_dens</code>	Loại	Đây là đại lượng dẫn xuất từ các biến sao chủ gốc. Giữ chúng cùng <code>st_teff</code> , <code>st_rad</code> , <code>st_mass</code> sẽ trùng thông tin và làm hệ số OLS khó diễn giải.
<code>rastr</code> , <code>ra</code> , <code>decstr</code> , <code>dec</code> , <code>glat</code> , <code>glon</code> , <code>elat</code> , <code>elon</code> , <code>sy_pm</code> , <code>sy_pmra</code> , <code>sy_pmdec</code>	Loại	Đây là vị trí và chuyển động riêng trên bầu trời. Chúng mô tả hệ quy chiếu quan sát từ Trái Đất, không quyết định trực tiếp bán kính hành tinh.
Các cột quang học <code>sy_bmag</code> , <code>sy_vmag</code> , <code>sy_jmag</code> , <code>sy_hmag</code> , <code>sy_kmag</code> , <code>sy_umag</code> , <code>sy_gmag</code> , <code>sy_rmag</code> , <code>sy_imag</code> , <code>sy_zmag</code> , <code>sy_w1mag</code> , <code>sy_w2mag</code> , <code>sy_w3mag</code> , <code>sy_w4mag</code> , <code>sy_gaiamag</code> , <code>sy_icmag</code> , <code>sy_tmag</code> , <code>sy_kepmag</code>	Loại	Các băng tần này đo độ sáng biểu kiến qua nhiều bộ lọc. Chúng tương quan mạnh với nhau và với khoảng cách, dễ tạo nhiễu photometry và đa cộng tuyến cục bộ.
<code>pl_nnotes</code> , <code>st_nphot</code> , <code>st_nrv</code> , <code>st_nspec</code> , <code>pl_nespec</code> , <code>pl_ntranspec</code> , <code>pl_ndispec</code>	Loại	Đây là số lượng ghi chú hoặc số lượng chuỗi đo trong archive. Chúng phản ánh độ phong phú dữ liệu quan sát, không phải cơ chế vật lý tạo nên bán kính hành tinh.

Ngược lại, `planet.csv` giữ lại 16 cột có ý nghĩa vật lý trực tiếp hơn. Bảng 12 tóm tắt vai trò của từng cột được giữ.

Bảng 12: Các cột được giữ trong `planet.csv` và vai trò mô hình hóa

Cột giữ lại	Nhóm	Lý do giữ lại
<code>pl_orbper</code>	Quỹ đạo	Chu kỳ quỹ đạo phản ánh điều kiện động lực học quanh sao chủ. Được giữ ban đầu nhưng sau đó bị VIF loại khi thông tin trùng với <code>pl_orbsmax</code> .
<code>pl_orbsmax</code>	Quỹ đạo	Bán trục lớn mô tả khoảng cách quỹ đạo, ảnh hưởng đến bức xạ nhận được và điều kiện hình thành hành tinh.
<code>pl_orbeccen</code>	Quỹ đạo	Độ lệch tâm mô tả hình dạng quỹ đạo, có thể liên quan đến lịch sử động lực của hệ.
<code>pl_trandur</code>	Transit	Thời lượng transit chứa thông tin về hình học transit và kích thước/quỹ đạo tương đối.
<code>pl_trandep</code>	Transit	Độ sâu transit liên quan đến tỷ lệ diện tích che khuất, nên có tín hiệu dự báo bán kính.
<code>pl_imppar</code>	Transit	Impact parameter mô tả vị trí đường đi của hành tinh qua đĩa sao, giúp hiệu chỉnh tín hiệu transit.
<code>pl_eqt</code>	Năng lượng	Nhiệt độ cân bằng đại diện môi trường nhiệt của hành tinh. Được giữ ban đầu nhưng sau đó bị VIF loại do trùng thông tin với quỹ đạo/nhiệt độ sao.
<code>pl_insol</code>	Năng lượng	Thông lượng bức xạ nhận từ sao chủ, giữ lại vì phản ánh điều kiện năng lượng tác động lên hành tinh.
<code>pl_bmasse</code>	Hành tinh	Khối lượng hành tinh là predictor vật lý quan trọng nhất để dự đoán bán kính.
<code>st_teff</code>	Sao chủ	Nhiệt độ sao chủ ảnh hưởng đến bức xạ và môi trường quanh hành tinh.
<code>st_rad</code>	Sao chủ	Bán kính sao chủ liên quan trực tiếp đến diễn giải tín hiệu transit.
<code>st_mass</code>	Sao chủ	Khối lượng sao chủ được giữ tạm để pipeline đánh giá, sau đó bị loại ở bước đầu do chồng lấp thông tin.
<code>st_met</code>	Sao chủ	Kim loại tính có thể liên quan đến thành phần vật chất và lịch sử hình thành hệ hành tinh.
<code>st_logg</code>	Sao chủ	Trọng lực bề mặt sao được giữ tạm để pipeline đánh giá, sau đó bị loại ở bước đầu vì là đại lượng dẫn xuất/chồng lấp.
<code>sy_dist</code>	Hệ sao	Khoảng cách tới hệ sao giúp mô tả thiên lệch quan sát và độ tin cậy tín hiệu.
<code>pl_rade</code>	Target	Bán kính hành tinh theo đơn vị Trái Đất, là biến mục tiêu cần dự đoán.

Sau hai giai đoạn này, `planet.csv` là tập dữ liệu đã được chọn lọc trước pipeline, gồm các biến vật lý có ý nghĩa hơn cho bài toán dự đoán `pl_rade`. Cụ thể, file này giữ các nhóm biến quỹ đạo, transit, năng lượng, khối lượng hành tinh, đặc trưng sao chủ và khoảng cách hệ sao.

Phân biệt với bước VIF trong pipeline. VIF không thuộc quá trình tạo `planet.csv`. VIF chỉ được tính sau khi pipeline đã đọc `planet.csv`, áp dụng domain restriction, chia train/test, log-transform, Winsorization và MICE imputation. Nói cách khác, giai đoạn 1–2 là chọn biến trước pipeline; còn VIF là thanh lọc tự động bên trong pipeline trên tập train đã xử lý.

Trong pipeline chính, hai biến `st_mass` và `st_logg` được loại ở bước đầu vì thông tin chồng lấp mạnh với các biến sao chủ còn lại. Sau đó VIF tiếp tục loại các biến đa cộng tuyến vượt ngưỡng như trình bày ở Bảng 17.

2.3.2 Làm sạch dữ liệu (Clean Data)

Pipeline tuân thủ nguyên tắc tránh data leakage: chia train/test trước, sau đó chỉ fit tham số tiền xử lý trên train và áp dụng lại cho test.

Load data → Domain restriction → Train/test split,

Fit preprocessing trên train → Transform test bằng tham số train. (32)

```

1 raw = raw[(raw["pl_rade"] <= 1.6) & (raw["pl_bmasse"] <= 10)]
2 train_raw, test_raw = train_test_split_frame(
3     raw, test_size=0.2, random_state=RANDOM_STATE
4 )
5
6 pipeline = DataPipeline(target="pl_rade")
7 X_train, y_train = pipeline.fit_transform(train_raw)
8 X_test, y_test = pipeline.transform(test_raw, include_target=True)

```

Đoạn mã 8: Tách train/test trước khi fit preprocessing

Sau khi chia dữ liệu, train có 867 dòng và test có 217 dòng. Tất cả các bước log-transform, Winsorization, MICE, VIF và standardization đều được fit trên train.

2.3.3 Xây dựng dữ liệu (Construct Data)

Log-transform. Nhiều đại lượng thiên văn có quan hệ lũy thừa, ví dụ Định luật Kepler hoặc quan hệ bức xạ. Với quan hệ:

$$y = Cx^{\alpha}, \quad (33)$$

lấy log hai vế đưa bài toán về dạng gần tuyến tính:

$$\log y = \log C + \alpha \log x. \quad (34)$$

Pipeline dùng phép biến đổi:

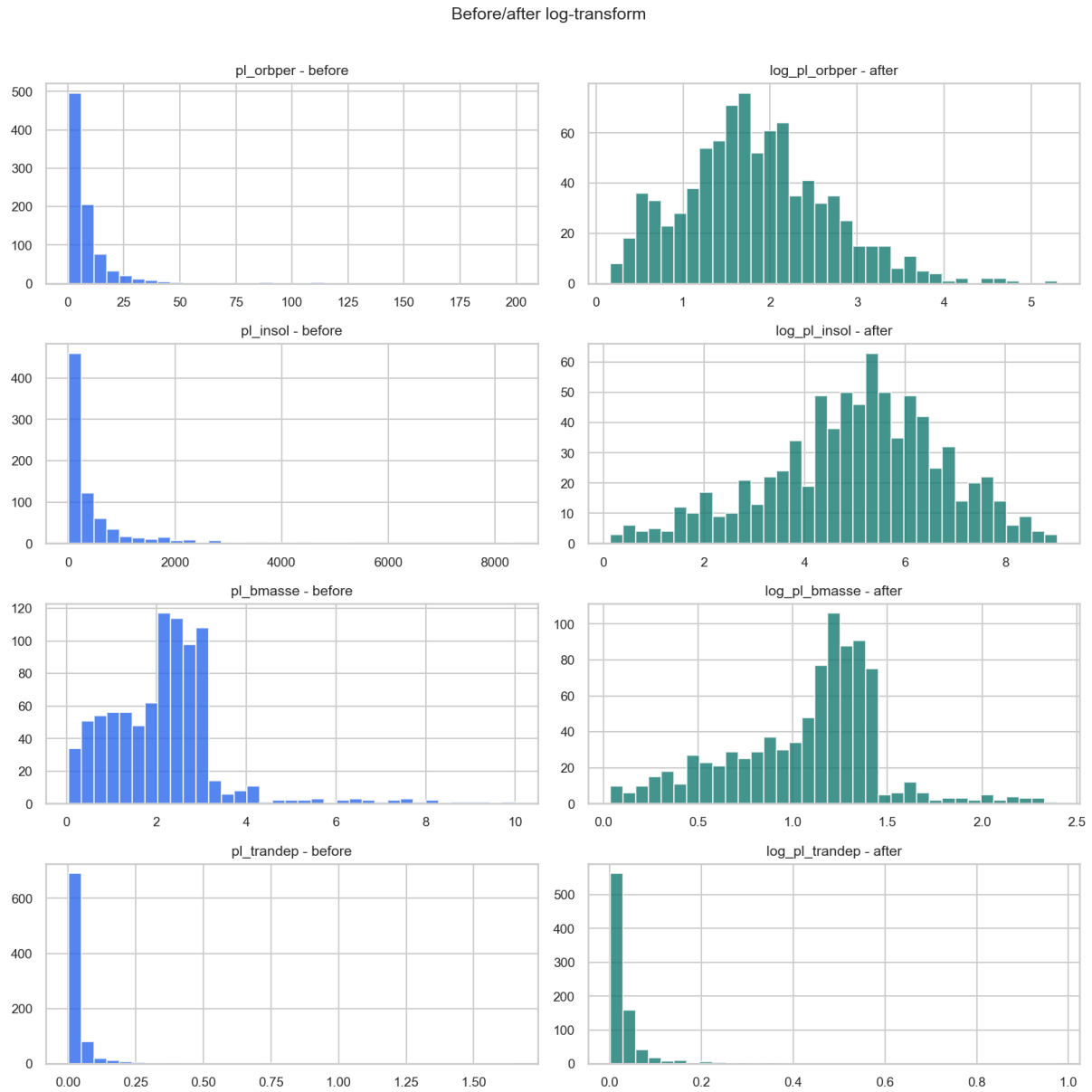
$$z = \log(1 + x), \quad (35)$$

và target được chuyển thành:

$$y = \log(1 + \text{pl_rade}) = \log_pl_rade. \quad (36)$$

Bảng 13: Ảnh hưởng của log-transform lên skewness

Biến gốc	Biến mới	Missing	Min trước	Max trước	Skew trước	Skew sau
pl_orbper	log_pl_orbper	0	0.1769	199.6688	6.8615	0.4901
pl_orbsmax	log_pl_orbsmax	73	0.0050	0.6193	3.2313	2.5948
pl_trandep	log_pl_trandep	40	0.0012	1.6536	10.1802	6.9522
pl_insol	log_pl_insol	83	0.1440	8385.9000	3.9596	-0.3815
pl_bmasse	log_pl_bmasse	0	0.0364	10.0000	1.7694	-0.2716
pl_eqt	log_pl_eqt	67	171.7000	3186.0000	0.8788	-0.4766
sy_dist	log_sy_dist	6	6.8693	2712.5800	1.1829	-1.1509
pl_rade	log_pl_rade	0	0.3098	1.6000	-0.6930	-0.9918



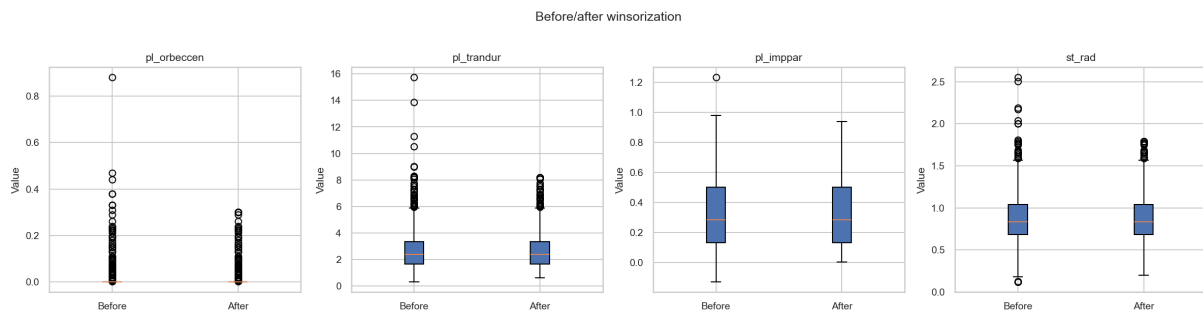
Hình 9: So sánh phân phối trước và sau log-transform. Các biến có đuôi phải dài được nén thang đo rõ rệt, đặc biệt là `pl_orbper`, `pl_insol` và `pl_bmasse`.

Winsorization. Nhóm không xóa ngoại lai mà cap giá trị ở hai đầu phân phối theo bách phân vị 1% và 99%:

$$x' = \begin{cases} q_{0.01}, & x < q_{0.01}, \\ x, & q_{0.01} \leq x \leq q_{0.99}, \\ q_{0.99}, & x > q_{0.99}. \end{cases} \quad (37)$$

Bảng 14: Tóm tắt Winsorization trên tập train

Cột	p01	p99	Dưới p01	Trên p99	Clipped (%)
pl_orbeccen	0.0000	0.2994	0	8	1.061
pl_trandur	0.6241	8.1895	9	9	2.120
pl_imppar	0.0040	0.9395	9	9	2.179
st_rad	0.1963	1.7887	9	9	2.076



Hình 10: Boxplot trước và sau Winsorization. Các điểm cực trị được kéo về ngưỡng fit trên train, trong khi phần trung tâm của phân phối được giữ nguyên.

MICE imputation. Sau log-transform và Winsorization, tập train vẫn còn missing values ở 9 cột. Nhóm dùng MICE (Multiple Imputation by Chained Equations) thay cho median imputation đơn giản để bảo toàn cấu trúc tương quan đa biến.

Bảng 15: Missing values sau log-transform trên tập train

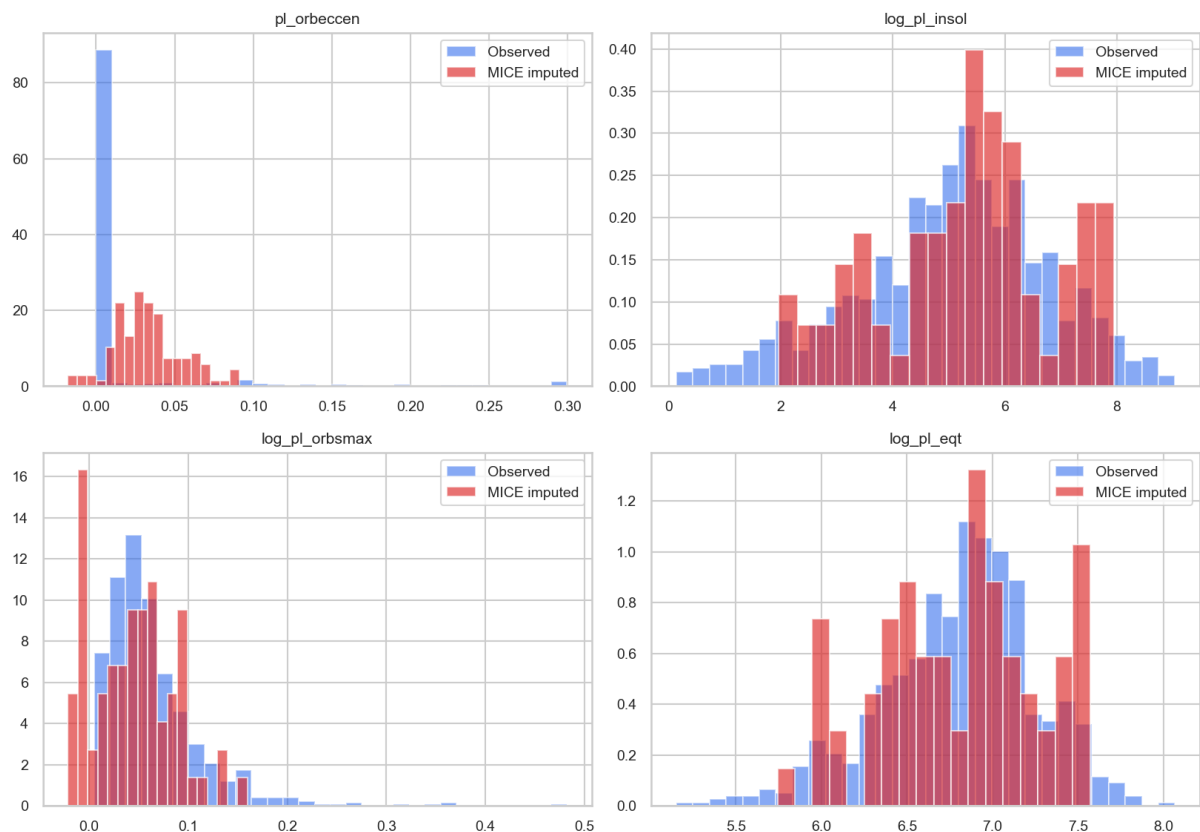
Cột	Missing count	Missing (%)	Kiểu dữ liệu
pl_orbeccen	113	13.033	float64
log_pl_insol	83	9.573	float64
log_pl_orbsmax	73	8.420	float64
log_pl_eqt	67	7.728	float64
pl_imppar	41	4.729	float64
log_pl_trandep	40	4.614	float64
st_met	24	2.768	float64
pl_trandur	18	2.076	float64
log_sy_dist	6	0.692	float64

MICE trong pipeline dùng 5 bản imputation, 5 vòng chained iterations, hồi quy Ridge phụ với regularization 0.001 và noise scale 0.15.

Bảng 16: Tóm tắt giá trị được MICE impute trên tập train

Cột	Missing	Missing (%)	Median khởi tạo	Observed mean	Imputed mean	Imputed std
pl_orbeccen	113	13.033	0.000000	0.013696	0.033514	0.021444
log_pl_insol	83	9.573	5.207544	5.036231	5.332386	1.555943
log_pl_orbsmax	73	8.420	0.052213	0.064713	0.045031	0.040655
log_pl_eqt	67	7.728	6.856987	6.806412	6.805654	0.457823
pl_imppar	41	4.729	0.284700	0.346475	0.394104	0.059489
log_pl_trandep	40	4.614	0.020498	0.036569	0.046997	0.040054
st_met	24	2.768	0.000000	-0.025205	-0.097143	0.039923
pl_trandur	18	2.076	2.372000	2.699806	2.127761	1.064210
log_sy_dist	6	0.692	6.104394	5.830641	5.143891	0.991066

Observed vs MICE-imputed distributions



Hình 11: So sánh phân phối observed và phân được MICE impute. Giá trị impute không bị gom về một điểm trung vị duy nhất mà vẫn có độ phân tán theo quan hệ đa biến.

Giai đoạn 3 trong DataPipeline: loại biến bằng VIF. Khác với hai giai đoạn tạo `planet.csv` ở trên, bước VIF diễn ra bên trong DataPipeline. Sau khi train/test đã được tách và tập train đã đi qua log-transform, Winsorization và MICE, pipeline kiểm tra đa cộng tuyến bằng hệ số

phóng đại phương sai:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (38)$$

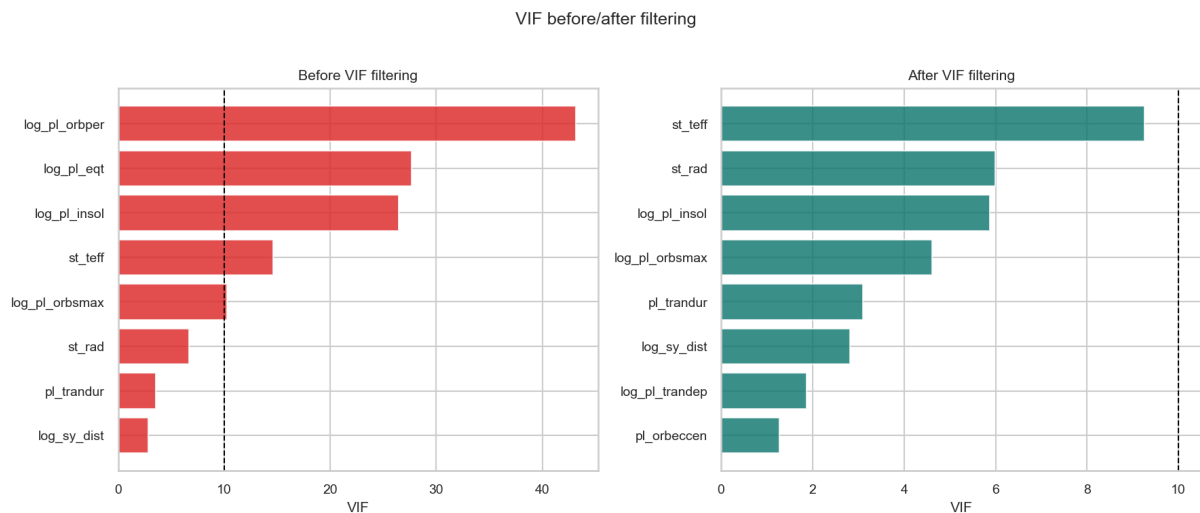
trong đó R_j^2 là hệ số xác định khi hồi quy biến x_j theo các biến còn lại. Ngưỡng loại biến là $\text{VIF} > 10$.

Bảng 17: Lịch sử lọc VIF

Biến có VIF cao nhất	VIF	Quyết định
log_pl_orbper	43.1374	Drop
log_pl_eqt	20.4514	Drop
st_teff	9.2570	Dừng vì nhỏ hơn 10

Bảng 18: VIF cuối cùng của các feature còn lại

Feature	VIF	Feature	VIF
st_teff	9.2570	log_pl_trandep	1.8730
st_rad	5.9915	pl_orbeccen	1.2736
log_pl_insol	5.8778	st_met	1.1861
log_pl_orbsmax	4.6054	pl_imppar	1.0732
pl_trandur	3.0931	log_pl_bmasse	1.0598
log_sy_dist	2.8147		



Hình 12: VIF trước và sau lọc. Việc loại log_pl_orbper và log_pl_eqt đưa tất cả VIF còn lại xuống dưới ngưỡng 10.

Sau VIF filtering, 11 feature cuối cùng là:

pl_orbeccen	pl_trandur	pl_imppar
st_teff	st_rad	st_met
log_pl_orbsmax	log_pl_trandep	log_pl_insol
log_pl_bmasse	log_sy_dist	

Chuẩn hóa dữ liệu. Tất cả feature cuối cùng được chuẩn hóa bằng z-score:

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}, \quad (39)$$

trong đó μ_j^{train} và σ_j^{train} chỉ được fit trên train.

2.4 Xây Dựng Mô Hình (Modeling)

Bốn mô hình tuyến tính chính được so sánh:

1. **OLS full:** dùng toàn bộ 11 feature sau tiền xử lý.
2. **OLS selected:** chọn feature có p-value nhỏ hơn 0.05 từ OLS full.
3. **Ridge:** dùng hàm `ridge_fit` ở Phần 1 và chọn λ bằng 5-fold CV.
4. **Lasso:** dùng hàm `lasso_fit` ở Phần 1 và chọn λ bằng 3-fold CV.

2.4.1 OLS full và chọn biến theo p-value

OLS full có $R_{\text{train}}^2 = 0.681784$ và $R_{\text{adj,train}}^2 = 0.677690$. Các biến có ý nghĩa thống kê ở mức 0.05 được giữ lại cho OLS selected:

pl_trandur, pl_imppar, st_teff, log_pl_trandep, log_pl_bmasse, log_sy_dist.

Bảng 19: Bảng suy diễn hệ số OLS full

Feature	Coef	Std.Err.	t-stat	p-value	CI lower	CI upper
pl_orbeccen	-0.004508	0.002704	-1.6673	0.095816	-0.009815	0.000799
pl_trandur	-0.011414	0.004214	-2.7088	0.006888	-0.019684	-0.003144
pl_imppar	0.005751	0.002482	2.3169	0.020743	0.000879	0.010622
st_teff	-0.014611	0.007290	-2.0043	0.045348	-0.028918	-0.000303
st_rad	0.009279	0.005864	1.5822	0.113964	-0.002231	0.020790
st_met	0.000139	0.002609	0.0533	0.957505	-0.004982	0.005260
log_pl_orbsmax	0.007766	0.005142	1.5104	0.131315	-0.002326	0.017857
log_pl_trandep	0.013692	0.003279	4.1757	3.28×10^{-5}	0.007256	0.020128
log_pl_insol	-0.005905	0.005809	-1.0165	0.309668	-0.017305	0.005496
log_pl_bmasse	0.095317	0.002466	38.6447	8.78×10^{-190}	0.090476	0.100158
log_sy_dist	0.034443	0.004020	8.5689	4.84×10^{-17}	0.026554	0.042332

Biến $\log_pl_bmasse$ có hệ số dương lớn nhất và p-value cực nhỏ, phù hợp với trực giác vật lý: hành tinh có khối lượng lớn thường có bán kính lớn hơn.

2.4.2 Chọn siêu tham số cho Ridge và Lasso

Ridge quét lưới λ từ 10^{-3} đến 10^3 theo thang log. Giá trị tốt nhất theo CV-MSE là:

$$\lambda_{\text{Ridge}}^* = 10. \quad (40)$$

Bảng 20: Một số kết quả Ridge CV

λ	Mean CV-MSE	Std CV-MSE	Mean CV- R^2
0.0010	0.005075	0.001660	0.673407
0.1000	0.005075	0.001659	0.673410
1.0000	0.005075	0.001656	0.673433
3.1623	0.005074	0.001650	0.673469
10.0000	0.005074	0.001629	0.673433
31.6228	0.005090	0.001571	0.672223
100.0000	0.005240	0.001432	0.661976
1000.0000	0.008594	0.001069	0.441426

Sau khi mở rộng lưới, Lasso quét 13 giá trị $\lambda \in [10^{-4}, 10^2]$ theo thang log bằng 3-fold CV. Lưới mới cho thấy vùng regularization tốt khá phẳng ở các λ nhỏ, sau đó CV-MSE bắt đầu tăng rõ khi $\lambda \geq 0.3162$. Giá trị tốt nhất là:

$$\lambda_{\text{Lasso}}^* = 0.0316228. \quad (41)$$

Bảng 21: Kết quả Lasso CV

λ	Mean CV-MSE	Std CV-MSE	Mean CV- R^2
0.0001	0.005093	0.001421	0.672720
0.0003	0.005093	0.001421	0.672720
0.001	0.005093	0.001421	0.672720
0.003	0.005093	0.001421	0.672722
0.010	0.005093	0.001420	0.672726
0.032	0.005092	0.001420	0.672730
0.100	0.005093	0.001418	0.672675
0.316	0.005106	0.001415	0.671822
1.000	0.005140	0.001420	0.669538
3.162	0.005301	0.001387	0.658832
10.000	0.005953	0.001265	0.615589
31.623	0.008702	0.000978	0.434213
100.000	0.015532	0.001217	-0.012201

Với $\lambda^* = 0.0316228$, Lasso vẫn không triệt tiêu hệ số nào: 0/11 feature có hệ số bằng 0 theo ngưỡng 10^{-10} . Điều này cho thấy Lasso trong lần chạy này hoạt động giống một OLS được làm mượt nhẹ hơn là một bộ chọn biến thưa. Lưới mở rộng cũng làm rõ rằng nếu phạt mạnh hơn

($\lambda \geq 1$), sai số CV tăng dần; do đó việc không chọn nghiệm thưa là kết quả thực nghiệm của dữ liệu, không phải do lưới ban đầu quá hẹp.

2.5 Dự Báo (Prediction)

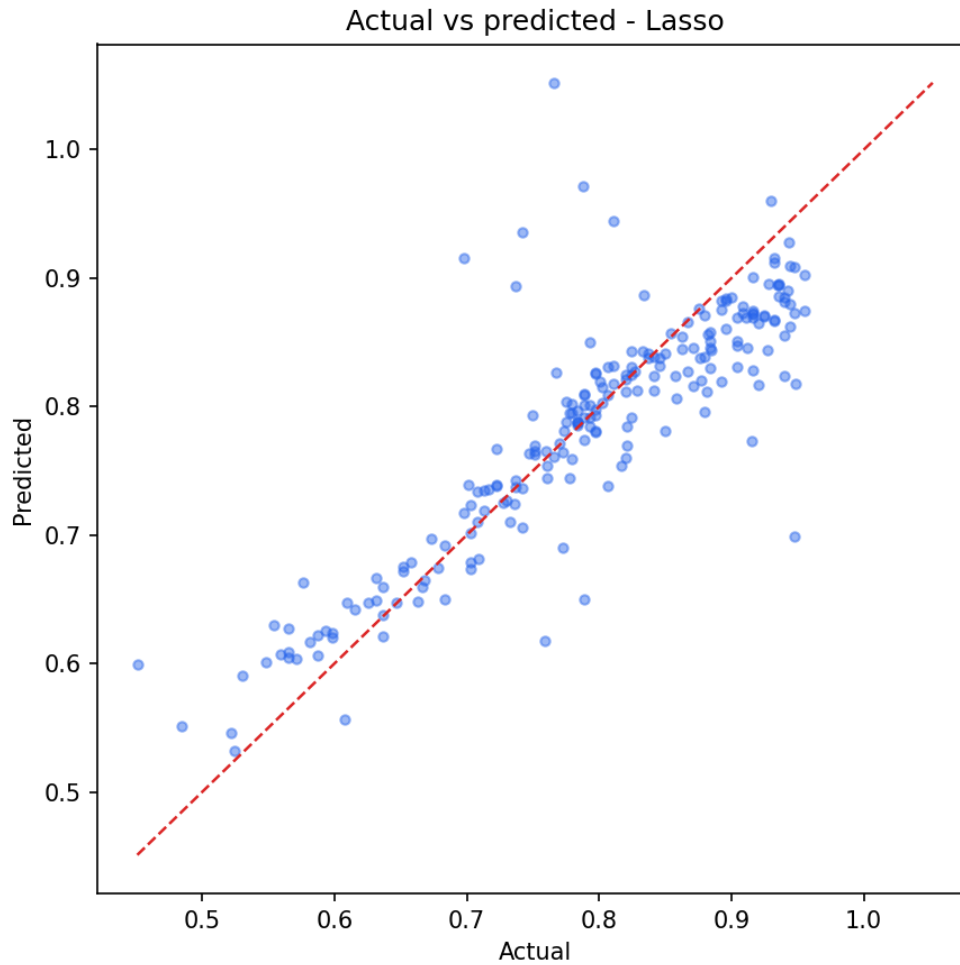
Mô hình tuyến tính tốt nhất trên test set là Lasso. Vì target được huấn luyện trên thang $\log_{pl_rade}$, dự báo được biến đổi ngược bằng:

$$\widehat{pl_rade} = \exp(\hat{y}) - 1. \quad (42)$$

Bảng 22: Một số dự báo của mô hình Lasso trên tập test

STT	Actual log	Pred log	Residual log	Actual radius	Pred radius	APE (%)
1	0.703098	0.673404	0.029694	1.020000	0.960901	5.794
2	0.774727	0.788004	-0.013276	1.170000	1.199002	2.479
3	0.904218	0.850695	0.053524	1.470000	1.341272	8.757
4	0.751416	0.762798	-0.011382	1.120000	1.144267	2.167
5	0.797507	0.797699	-0.000192	1.220000	1.220426	0.035
6	0.548121	0.601355	-0.053234	0.730000	0.824590	12.958
7	0.581098	0.616384	-0.035286	0.788000	0.852218	8.150
8	0.943906	0.879492	0.064414	1.570000	1.409674	10.212
9	0.783902	0.787921	-0.004019	1.190000	1.198819	0.741
10	0.662688	0.647882	0.014805	0.940000	0.911489	3.033

Trên thang bán kính gốc, Lasso đạt $R^2 = 0.712521$, $RMSE = 0.130353$, $MAE = 0.085233$ và $MAPE = 7.038\%$. Các chỉ số này không dùng để chọn mô hình chính, nhưng giúp diễn giải sai số theo đơn vị bán kính Trái Đất.



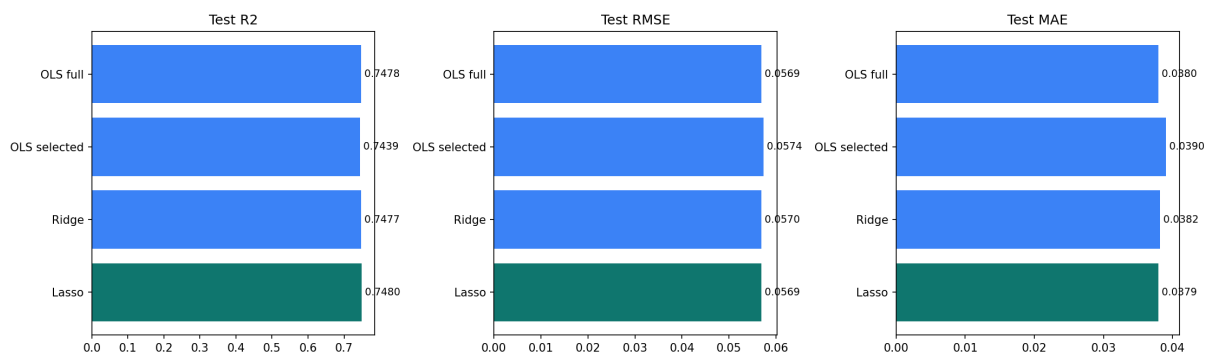
Hình 13: Actual vs Predicted trên tập test của mô hình Lasso. Các điểm càng gần đường chéo thì dự báo càng sát giá trị thật.

2.6 Đánh Giá Mô Hình (Evaluation)

2.6.1 So sánh mô hình tuyến tính

Bảng 23: So sánh các mô hình trên tập test

Mô hình	Test R^2	Test RMSE	Test MAE	CV-MSE	CV- R^2
Lasso	0.748011	0.056917	0.037929	0.005092	0.672730
OLS full	0.747794	0.056941	0.037952	0.005075	0.673407
Ridge	0.747678	0.056954	0.038204	0.005074	0.673433
OLS selected	0.743913	0.057378	0.039013	0.005103	0.671611



Hình 14: So sánh Test R^2 , RMSE và MAE giữa các mô hình tuyến tính. Lasso đạt Test R^2 cao nhất, nhưng khác biệt giữa Lasso, OLS full và Ridge rất nhỏ.

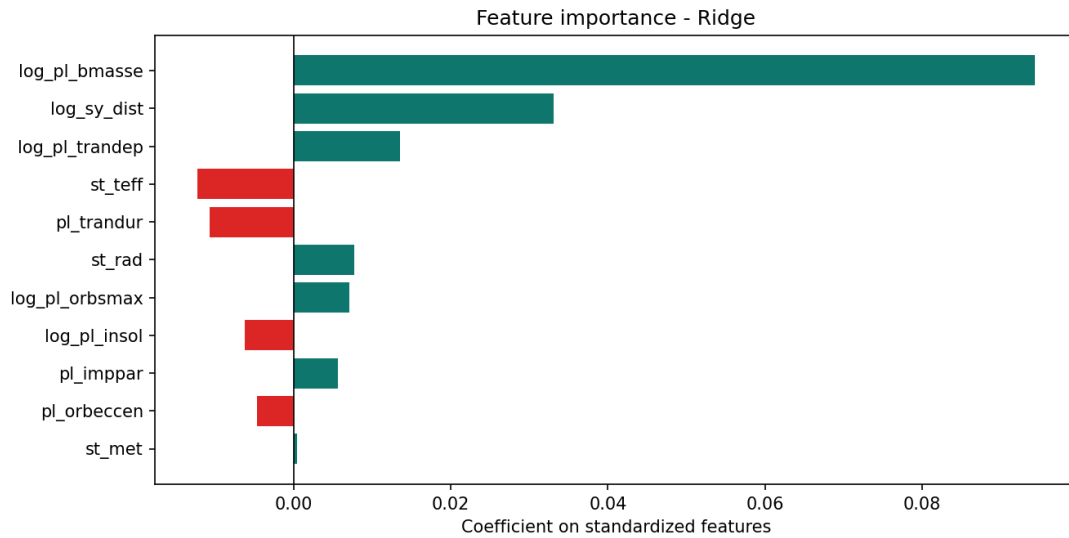
Nhận xét:

- Lasso có Test R^2 cao nhất là 0.748011 và Test RMSE thấp nhất là 0.056917.
- OLS full gần như tương đương Lasso, cho thấy quan hệ tuyến tính sau tiền xử lý đã mô tả dữ liệu khá tốt.
- Ridge có CV-MSE thấp nhất trong nhóm mô hình tuyến tính, phản ánh regularization giúp ổn định hơn trên các fold train.
- OLS selected ít biến hơn nhưng Test R^2 giảm nhẹ xuống 0.743913; việc rút gọn biến giúp dễ diễn giải hơn nhưng có thể mất một phần thông tin dự báo.

Sự khác biệt giữa Lasso, OLS full và Ridge rất nhỏ: Test R^2 của ba mô hình chỉ chênh nhau ở chữ số thập phân thứ tư. Lý do chính là pipeline đã làm phần việc mà regularization thường hỗ trợ: log-transform giảm lệch phải, Winsorization giảm ảnh hưởng cực trị, và VIF filtering đã loại log_pl_orbper, log_pl_eqt để đưa VIF cuối cùng xuống dưới 10. Khi đa cộng tuyến đã được xử lý trước, Ridge không còn lợi thế ổn định hệ số quá lớn, còn Lasso với $\lambda^* = 0.0316228$ vẫn không tạo nghiệm thừa. Vì vậy mô hình tuyến tính cơ bản đã gần đạt hiệu năng tối đa của nhóm mô hình tuyến tính trên tập feature hiện có.

Mặt khác, $R^2_{\text{test}} \approx 0.748$ cũng có nghĩa khoảng 25.2% phương sai của log_pl_rade vẫn chưa được giải thích. Phần còn lại có thể đến từ ba nguồn: quan hệ vật lý phi tuyến giữa khối lượng–bán kính ở từng cấu tạo hành tinh, thiếu biến mô tả thành phần bên trong/tuổi hệ sao/độ chính xác phép đo, và selection bias của dữ liệu quan trắc thiên văn. Do đó, kết quả này nên được đọc như một mô hình nội suy tuyến tính tốt cho nhóm terrestrial/Super-Earth, không phải một phương trình vật lý đầy đủ của quá trình hình thành hành tinh.

2.6.2 Phân tích hệ số và feature importance

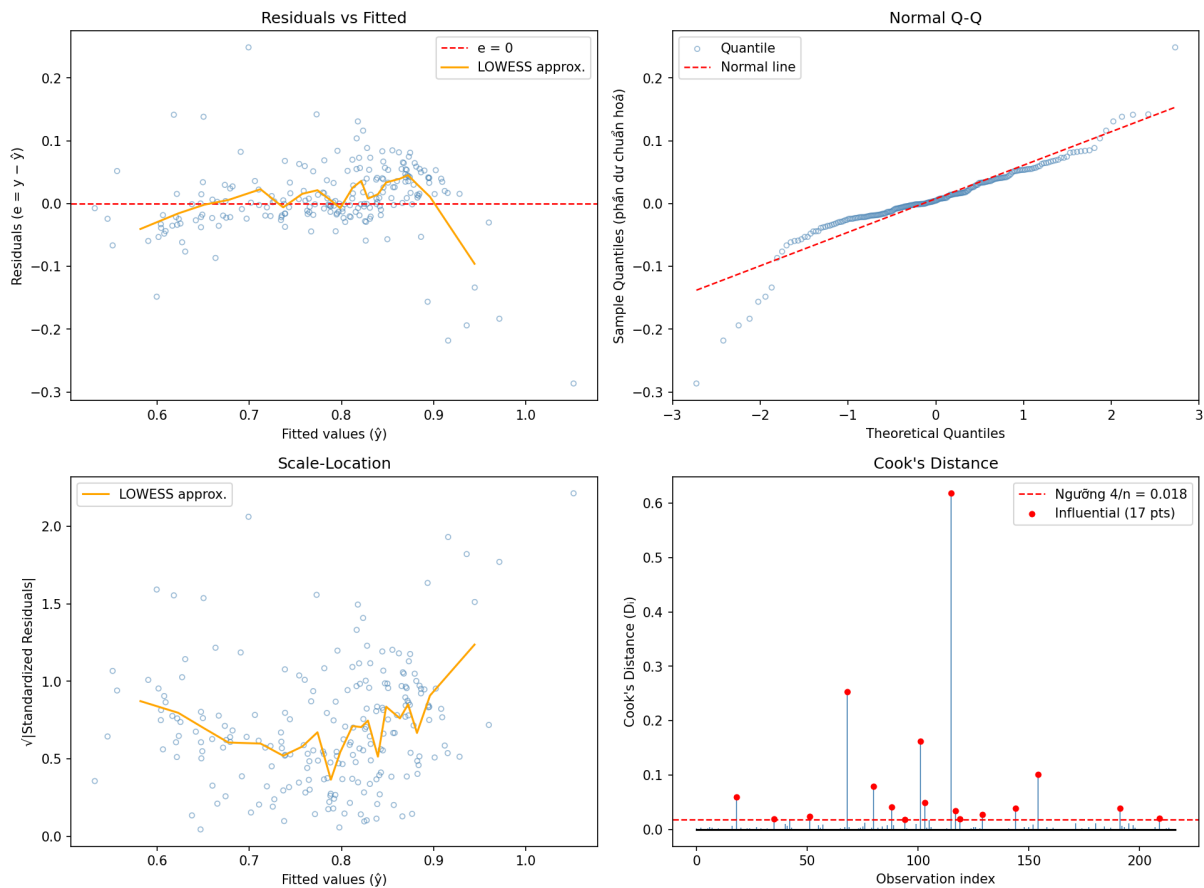


Hình 15: Feature importance dựa trên hệ số Ridge sau chuẩn hóa. Vì các feature đã được standardize, độ lớn hệ số có thể so sánh trực tiếp hơn.

Các biến quan trọng nhất gồm `log_pl_bmasse`, `log_sy_dist` và `log_pl_trandep`. Trong đó, `log_pl_bmasse` là biến có ảnh hưởng dương mạnh nhất, phù hợp với quan hệ vật lý giữa khối lượng và bán kính hành tinh. `log_sy_dist` cũng có hệ số dương khá lớn, nhưng không nên diễn giải theo nghĩa nhân quả rằng hành tinh ở xa Trái Đất thì tự nhiên có bán kính lớn hơn. Cách đọc hợp lý hơn là *selection bias*: ở khoảng cách xa, các kính thiên văn dễ phát hiện và xác nhận những hành tinh có tín hiệu mạnh hơn, thường là transit sâu hơn hoặc bán kính lớn hơn. Vì vậy hệ số này phản ánh thiên lệch quan sát của catalog nhiều hơn là một cơ chế vật lý trực tiếp.

2.6.3 Phân tích phần dư và giả định mô hình

Phân Tích Phần Dư (Residual Diagnostic Plots)



Hình 16: Bốn biểu đồ chẩn đoán phần dư của mô hình Lasso: Residuals vs Fitted, Normal Q-Q, Scale-Location và Cook's Distance.

Nhận xét từ residual plots:

- Phần dư tập trung quanh 0, cho thấy mô hình không có thiên lệch hệ thống quá rõ trên đa số quan sát.
- Q-Q plot cho thấy phần giữa của phân phối phần dư tương đối gần chuẩn, nhưng hai đuôi vẫn lệch khỏi đường chéo; đây là dấu hiệu dữ liệu còn ngoại lai hoặc quan hệ phi tuyến.
- Cook's Distance lớn nhất được ghi nhận là 0.617979. Với $n = 217$ mẫu test, ngưỡng $4/n \approx 0.01843$ đánh dấu 17 điểm influential.

Bảng 24: Kiểm định chẩn đoán trên phần dư test của Lasso

Kiểm định	Statistic	p-value	Diễn giải
Shapiro-Wilk	0.887620	1.21×10^{-11}	Bác bỏ giả thuyết phần dư phân phối chuẩn; hai đuôi còn lệch, phù hợp với Q-Q plot.
Breusch-Pagan	36.895113	0.000120	Bác bỏ giả thuyết phương sai không đổi; vẫn còn heteroskedasticity sau log-transform và Winsorization.

Nhóm không xóa 17 điểm influential vì đây là dữ liệu thiên văn thật, không phải lỗi nhập liệu đã được chứng minh. Các điểm này có thể đại diện cho hệ hành tinh hiếm, sai số đo lớn, hoặc vùng biên của catalog. Xóa cứng sẽ làm báo cáo đẹp hơn về mặt diagnostic nhưng dễ tạo thiên lệch mẫu. Vì vậy nhóm giữ lại chúng, dùng log-transform/Winsorization để giảm ảnh hưởng cực trị, và ghi nhận đây là một giới hạn khi diễn giải hệ số.

2.6.4 Kiểm chứng giả thuyết Giãn nở nhiệt (Thermal Expansion Hypothesis)

Kiểm định tác động tương tác giữa Khối lượng và Nhiệt độ: Từ hiện tượng đường LOWESS bị cong nhẹ ở vùng dự đoán cao trên biểu đồ chẩn đoán Residuals vs Fitted, giả thuyết được đặt ra là: tác động của khối lượng (lực nén hấp dẫn) lên cấu trúc bán kính hành tinh không độc lập, mà bị chi phối bởi mức độ bức xạ nhiệt (hiệu ứng giãn nở). Để kiểm chứng, mô hình hồi quy được mở rộng thành một cấu trúc phi tuyến cục bộ thông qua đại lượng tương tác:

$$Y = \beta_1 X_{\text{mass}} + \beta_2 X_{\text{heat}} + \beta_3 (X_{\text{mass}} \times X_{\text{heat}}) \quad (43)$$

Thực nghiệm đầu tiên tiến hành bổ sung biến tương tác giữa khối lượng hành tinh và nhiệt độ bề mặt của sao chủ ($\log_{\text{pl_bmasse}} \times \text{st_teff}$). Kết quả truy xuất cho thấy p-value của biến này đạt mức 0.098, vượt ngưỡng mức ý nghĩa $\alpha = 0.05$. Sự thất bại của giả thuyết này có thể lý giải bằng bản chất vật lý: nhiệt độ bề mặt sao chủ không phản ánh chính xác thông lượng nhiệt (heat flux) mà hành tinh thực nhận nếu khuyết đi tham số về khoảng cách quỹ đạo.

Để khắc phục, thực nghiệm thứ hai thay thế nhiệt độ sao bằng thông lượng bức xạ thực nhận ($\log_{\text{pl_insol}}$ tạo thành biến tương tác ($\log_{\text{pl_bmasse}} \times \log_{\text{pl_insol}}$). Kết quả ghi nhận p-value ≈ 0 , khẳng định sự tồn tại của cơ chế tương tác nhiệt - khối lượng trên tập huấn luyện. Tuy nhiên, nghịch lý xuất hiện khi kiểm định hiệu năng: hệ số xác định trên tập kiểm thử (R_{Test}^2) lại sụt giảm từ 0.747 xuống 0.741, đồng thời RMSE tăng.

Bản chất của hiện tượng này là minh chứng kinh điển cho bài toán *Sự đánh đổi giữa Độ chệch và Phương sai (Bias-Variance Tradeoff)*. Biến tương tác đã vô tình kích hoạt hiện tượng Đa cộng tuyến cấu trúc (Structural Multicollinearity). Mặt phẳng OLS bị ép phải vận xoắn

(giảm Bias) nhằm cố gắng nội suy khít với các biến thiên cục bộ sinh ra từ hiệu ứng giãn nở nhiệt trong tập huấn luyện, từ đó đánh mất khả năng khái quát hóa quy luật (tăng Variance) khi tiếp xúc với dữ liệu kiểm thử.

Kết luận: Nỗ lực triệt tiêu sai số phần dư cục bộ bằng các biến tương tác đã dẫn đến hiện tượng quá khớp (Overfitting). Hệ quả này củng cố tính đúng đắn của nguyên lý tinh giản Occam's Razor trong thiết kế Machine Learning: cấu trúc OLS thuần túy hiện tại (không chứa đại lượng tương tác) là điểm cân bằng toàn cục tối ưu nhất giữa năng lực giải thích hiện tượng và sức mạnh ngoại suy định lượng.

2.6.5 Phương pháp nâng cao: Kernel Ridge Regression

Ngoài các mô hình tuyến tính chính, nhóm thử Kernel Ridge Regression với RBF kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2). \quad (44)$$

Do Kernel Ridge cần ma trận Gram kích thước $n \times n$, pipeline dùng tối đa 200 mẫu train để giữ chi phí tính toán hợp lý. Hyperparameters được chọn trên validation tách từ train, sau đó mới đánh giá một lần trên test.

Bảng 25: Một số cấu hình Kernel Ridge theo validation RMSE

Alpha	Gamma	Validation RMSE
0.010	0.010	0.071636
0.100	0.010	0.075317
0.001	0.010	0.075770
0.001	0.001	0.080698
1.000	0.010	0.084010
0.010	0.001	0.089867
0.100	0.001	0.091168
0.010	0.050	0.091262

Bảng 26: Kết quả Kernel Ridge tốt nhất trên test set

Mô hình	Alpha	Gamma	Test R^2	RMSE	MAE
Kernel Ridge (RBF)	0.01	0.01	0.700766	0.062023	0.038842

Kết quả Kernel Ridge thấp hơn nhóm mô hình tuyến tính chính trong lần chạy `part2/real/`, nhưng đây không phải là một so sánh công bằng về năng lực cuối cùng của phương pháp. Giới

hạn 200 mẫu train là quyết định kỹ thuật có chủ ý vì Kernel Ridge phải lưu và giải hệ trên ma trận Gram $n \times n$, làm chi phí bộ nhớ và thời gian tăng xấp xỉ theo $O(n^2)$ đến $O(n^3)$ tùy bước giải hệ. Do đó, kết quả này chỉ cho thấy cấu hình Kernel Ridge thử nghiệm trong báo cáo chưa vượt mô hình tuyến tính; nó không đủ để kết luận Kernel Ridge nói chung kém hơn OLS/Ridge/Lasso trên bài toán này.

2.7 Kết Luận Phần 2

Phần 2 đã hoàn thành một pipeline data fitting đầy đủ cho dữ liệu thực tế:

- Dữ liệu được chọn từ NASA Exoplanet Archive, sau đó giới hạn vào nhóm hành tinh đá và Super-Earth với 1084 mẫu.
- Báo cáo đã kiểm tra duplicate, missing values, ngoại lai/nhiều và các khả năng conflict trong dữ liệu quan trắc.
- Pipeline tiền xử lý gồm log-transform, Winsorization, MICE imputation, VIF filtering và standardization; các tham số đều fit trên train để tránh data leakage.
- Các mô hình chính tái sử dụng code tự cài ở Phần 1: OLS, Ridge, Lasso, k-fold cross-validation và residual analysis.
- Trên test set, Lasso là mô hình tuyến tính tốt nhất với Test $R^2 = 0.748011$, RMSE = 0.056917 và MAE = 0.037929 trên thang log target.
- Khi biến đổi ngược về bán kính gốc, Lasso đạt RMSE = 0.130353, MAE = 0.085233 và MAPE = 7.038%, cho thấy sai số dự báo có thể diễn giải được theo đơn vị bán kính Trái Đất.

Nhìn chung, báo cáo Part 2 đã bám sát quy trình trong ví dụ tham khảo: có phần giới thiệu bài toán, hiểu dữ liệu, chuẩn bị dữ liệu, xây dựng mô hình, dự báo và đánh giá mô hình. Đồng thời, so với ví dụ, phần này bổ sung thêm các kỹ thuật phù hợp với dữ liệu thiên văn như MICE, VIF filtering, regularization và phân tích feature importance.

3 Kết luận

3.1 Tóm tắt kết quả đạt được

Kết thúc quá trình thực hiện đồ án, nhóm đã hoàn thành toàn diện 100% các mục tiêu đề ra ban đầu, bao gồm cả yêu cầu cốt lõi lẫn các phần mở rộng (điểm cộng). Các kết quả chính yếu bao gồm:

- **Cài đặt thành công Phép khử Gauss với độ ổn định cao:** Xây dựng thuật toán từ đầu (from scratch) bằng Python, áp dụng triệt để kỹ thuật Partial Pivoting. Thuật toán xử lý thành công mọi dạng ma trận (vuông, chữ nhật, suy biến) và đã được kiểm chứng tính ổn định số học tương đương với thư viện `numpy.linalg` thông qua việc triệt tiêu hiện tượng Swamping.
- **Phát triển hệ thống phân rã và chéo hóa ma trận (SVD):** Hoàn thiện thuật toán phân rã giá trị suy biến (Singular Value Decomposition) dựa trên nền tảng toán học của trị riêng và vector riêng. Thuật toán có khả năng phân rã và trích xuất đúng các ma trận U, Σ, V^T .
- **Xây dựng bộ công cụ phân tích hiệu năng (Benchmarking Suite):** Thiết lập thành công kịch bản đo lường tự động (`benchmark.py`) và hệ thống sinh dữ liệu (`data_gen.py`) để thu thập hàng ngàn điểm dữ liệu về thời gian thực thi và sai số tương đối (Residual Norm) trên đa dạng kích thước ma trận ($n \in \{50, 100, 200, 500, 1000\}$).
- **Phân tích sâu sắc tính bất ổn định số học:** Sử dụng thành công ma trận Hilbert H_n (hệ điều kiện kém) và ma trận SPD để bộc lộ hiện tượng khuếch đại sai số ở chuẩn dấu phẩy động IEEE 754. Qua đó, minh chứng rõ ràng mối liên hệ giữa Số điều kiện $\kappa_2(A)$ và mức độ "trôi dạt" của vector nghiệm.
- **Trực quan hóa Toán học sinh động bằng Manim:** Chuyển hóa các phương trình đại số tuyến tính khô khan thành Video hoạt hình (Animation) chất lượng cao. Video mô phỏng trực quan các phép biến đổi hình học của ma trận trong không gian, giúp giải thích trực diện bản chất của quá trình phân rã.
- **Quy trình phát triển phần mềm chuyên nghiệp:** Ứng dụng triệt để Git/GitHub trong quản lý mã nguồn nhóm, cấu trúc thư mục module hóa (`solvers`, `config`), kết hợp Jupyter Notebook và $\text{\LaTeX}$ để xuất bản báo cáo học thuật đạt chuẩn.
- Cài đặt thêm phương pháp lặp **Gauss-Seidel** với cơ chế tự động kiểm tra điều kiện hội tụ (ma trận chéo trội chặt hàng).
- Xây dựng kịch bản benchmark để đo lường thời gian thực thi trên các kích thước ma trận lớn ($n \in \{50, 100, 200, 500, 1000\}$). Vẽ đồ thị *log-log* so sánh và chứng minh thực nghiệm chi phí tính toán của các phương pháp trực tiếp hoàn toàn bám sát đường lý thuyết $O(n^3)$.
- Nghiên cứu thành công sự bất ổn định số học: Sử dụng **Số điều kiện $\kappa_2(A)$** kết hợp đối chiếu

giữa ma trận Hilbert H_n (ill-conditioned) và ma trận SPD ngẫu nhiên để bộc lộ hiện tượng khuếch đại sai số, giải thích sâu sắc giới hạn của hệ thống dấu phẩy động IEEE 754.

3.2 Bài học rút ra

Quá trình thực hiện đồ án không chỉ là việc hiện thực hóa các công thức toán học lên ngôn ngữ lập trình, mà còn là một hành trình giúp nhóm thay đổi tư duy về khoa học tính toán:

- **Sự khác biệt giữa Toán học lý thuyết và Tính toán số học:** Nhóm nhận ra rằng một thuật toán đúng trên giấy chưa chắc đã chạy đúng trên máy tính. Việc hiểu về cấu trúc lưu trữ số thực *double precision* là chìa khóa để giải thích tại sao $0.1 + 0.2$ không bao giờ bằng 0.3 tuyệt đối trong RAM.
- **Tư duy phản biện qua dữ liệu:** Thay vì chấp nhận sai số, nhóm đã học được cách đặt câu hỏi “Tại sao?”. Việc quan sát đường sai số bám sát ngưỡng 10^{-16} đã giúp nhóm tin tưởng vào thuật toán của mình hơn là việc chỉ nhìn vào kết quả nghiệm cuối cùng.
- **Kỹ năng tối ưu hóa quy trình làm việc:** Nhóm đã làm quen với việc tự động hóa (Automation) thông qua *latexmk* và các script *benchmark*. Bài học về việc “lười biếng một cách thông minh” – viết script để máy tính tự đo đạc hàng ngàn phép tính – đã giúp nhóm tiết kiệm rất nhiều thời gian.
- **Giá trị của việc chuẩn hóa (Standardization):** Việc thống nhất sử dụng `config.py` và quy chuẩn `list[list[float]]` ngay từ đầu là bài học xương máu về quản lý dự án, giúp việc ghép code giữa các thành viên diễn ra trơn tru mà không bị xung đột kiểu dữ liệu.

3.3 Khó khăn chuyên môn và các nghịch lý Toán học tính toán

Trong suốt quá trình thực hiện đồ án, rào cản lớn nhất của nhóm không nằm ở cú pháp lập trình, mà nằm ở sự sai khác giữa Toán học lý thuyết (trên giấy) và Giải tích số (trên máy tính). Dưới đây là bốn khó khăn cốt lõi và bài học nhận thức nhóm đã rút ra.

1. Giới hạn của chuẩn IEEE 754

Hiện tượng: Trong giai đoạn kiểm thử ban đầu, nhóm kỳ vọng sai số phần dư sẽ bằng 0.0 tuyệt đối. Tuy nhiên, khi giải các hệ phương trình chứa phân số (như $\frac{1}{3}$) hay số thập phân (như 0.1), sai số phần dư luôn trôi nổi ở mức 10^{-16} .

Bản chất Toán học: Đây không phải là lỗi thuật toán, mà là giới hạn biểu diễn tất yếu của chuẩn dấu phẩy động IEEE 754 (Double Precision) [6]. Các phân số vô hạn tuần hoàn trong hệ cơ số 2 bị cắt cụt ở bit thứ 53, sinh ra nhiễu làm tròn (round-off error). Ranh giới này được gọi là *Machine Epsilon*: $\epsilon_{\text{machine}} \approx 2.22 \times 10^{-16}$.

Giải quyết: Nhóm phải từ bỏ tư duy ”so sánh bằng 0” (exact equality) của Toán học. Thay vào đó, hằng số $\text{EPSILON} = 1\text{e-}12$ được thiết lập tập trung trong `config.py` làm ngưỡng dung sai (tolerance), cho phép các thuật toán phân rã SVD và Jacobi hội tụ một cách an toàn mà không bị ảnh hưởng bởi nhiễu máy tính.

2. Sai số phần dư trên ma trận Hilbert

Hiện tượng: Khi đưa ma trận Hilbert H_n (với $n \geq 50$) vào thuật toán Gauss, máy tính trả về vector nghiệm $\hat{x}$ sai lệch hoàn toàn so với nghiệm lý thuyết. Điều kỳ lạ là khi kiểm tra lại bằng sai số phần dư $\|A\hat{x} - b\|_2$, kết quả vẫn cực kỳ nhỏ (cỡ 10^{-15}) – máy tính báo ”đúng” nhưng nghiệm thực tế lại ”sai hoàn toàn”.

Bản chất Toán học: Nhóm đã vấp phải nghịch lý kinh điển của hệ điều kiện kém (ill-conditioned system). Ma trận Hilbert có số điều kiện $\kappa_2(A)$ khổng lồ, lên tới 10^{20} . Theo Định lý sai số [3]:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa_2(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

Chỉ cần một hạt nhiễu $\epsilon_{\text{machine}}$ rất yếu trong vế phải b , sai số thực tế (forward error) của nghiệm sẽ bị khuếch đại lên $\kappa_2(A)$ lần. Sai số phần dư nhỏ chỉ là một ảo ảnh toán học do không gian bị kéo dãn quá mức – đây là ”bẫy” đặc trưng của hệ ill-conditioned mà bất kỳ nhà giải tích số nào cũng phải cảnh giác.

Giải quyết: Sự kiện này buộc nhóm phải nhìn nhận lại giá trị thực sự của thuật toán SVD. Nhờ khả năng dùng nghịch đảo giả (pseudo-inverse) và cắt tỉa các giá trị suy biến tiệm cận 0, SVD đã chứng minh được đây là công cụ duy nhất ”miễn nhiễm” một phần với sự bùng nổ sai số trên hệ điều kiện kém.

3. Bức tường của hằng số $\mathcal{O}(n^3)$ và nút thắt hiệu năng Python

Hiện tượng: Trong lý thuyết, cả Khử Gauss và SVD đều có độ phức tạp $\mathcal{O}(n^3)$. Tuy nhiên, khi nhóm chạy benchmark bằng Python thuần ở $n = 500, 1000$, chương trình bị treo (Timeout > 60 s). Trong khi đó, `np.linalg.solve` của NumPy giải quyết $n = 1000$ chỉ trong 0.036 giây.

Bản chất Toán học và Kiến trúc: Độ phức tạp $\mathcal{O}(n^3)$ chỉ mô tả tốc độ tăng trưởng – nó che giấu một hằng số thực thi C rất lớn. Thuật toán SVD (Jacobi/QR) đòi hỏi nhiều vòng quét, phép tính lượng giác và căn bậc hai hơn hẳn Gauss, khiến $C_{\text{SVD}} \gg C_{\text{Gauss}}$. Bên cạnh đó, rào cản của trình thông dịch Python (interpreted) khiến phần cứng không thể song song hóa các phép nhân ma trận để phát huy tối đa khả năng của CPU.

Giải quyết: Dưới sự cho phép của giảng viên, nhóm đã linh hoạt chuyển sang sử dụng backend BLAS/LAPACK thông qua NumPy cho các kích thước n lớn. Quyết định này đảm bảo thu thập đủ số liệu để vẽ đồ thị Log-Log chuẩn xác và minh chứng được sự song song giữa đường thực nghiệm với đường tiệm cận $\mathcal{O}(n^3)$ lý thuyết.

4. Ranh giới hội tụ khắt khe của phương pháp lặp Gauss-Seidel

Hiện tượng: Ban đầu, khi thử nghiệm Gauss-Seidel trên các ma trận SPD ngẫu nhiên, thuật toán liên tục báo ERR (vượt quá số vòng lặp tối đa mà không hội tụ), mặc dù lý thuyết khẳng định Gauss-Seidel chắc chắn hội tụ trên ma trận SPD.

Bản chất Toán học: Định lý chỉ khẳng định ma trận SPD sẽ hội tụ, nhưng không hứa hẹn tốc độ hội tụ. Nếu bán kính phổ (spectral radius) của ma trận lặp $\rho(G)$ nằm quá sát 1, phương pháp lặp tiến triển cực kỳ chậm và cạn kiệt giới hạn vòng lặp cho phép trước khi đạt được ngưỡng hội tụ.

Giải quyết: Nhóm đã can thiệp vào khâu sinh dữ liệu (`data_gen.py`), bắt buộc mọi ma trận SPD phải được cộng thêm biên độ vào đường chéo chính để thỏa mãn điều kiện **chéo trội nghiêm ngặt** (Strictly Diagonally Dominant):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, \dots, n$$

Chỉ khi ép chặt điều kiện này, bán kính phổ $\rho(G)$ mới thực sự nhỏ hơn 1 đủ nhiều, giúp Gauss-Seidel hội tụ ổn định và phát huy sức mạnh tốc độ $\mathcal{O}(n^2)$ đặc trưng của phương pháp lặp.

Tài liệu

- [1] Gilbert Strang, *Introduction to Linear Algebra*, 5th Edition, Wellesley-Cambridge Press, 2016.
- [2] Gene H. Golub và Charles F. Van Loan, *Matrix Computations*, 4th Edition, Johns Hopkins University Press, 2013.
- [3] Lloyd N. Trefethen và David Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [4] Gilbert Strang, *MIT 18.06SC Linear Algebra*, MIT OpenCourseWare, <https://ocw.mit.edu/courses/18-06sc-linear-algebra-fall-2011/>, truy cập ngày 14/04/2026.
- [5] Virginia C. Klema và Alan J. Laub, *The singular value decomposition: Its computation and some applications*, IEEE Transactions on Automatic Control, Vol. 25, No. 2, pp. 164–176, 1980.
- [6] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, <https://ieeexplore.ieee.org/document/8766229>, truy cập ngày 09/04/2026.
- [7] Nicholas J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd Edition, SIAM, 2002.
- [8] David Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys, Vol. 23, No. 1, pp. 5–48, 1991.
- [9] NumPy Developers, *NumPy v1.26 Manual – Linear Algebra (numpy.linalg)*, <https://numpy.org/doc/stable/reference/routines.linalg.html>, truy cập ngày 09/04/2026.
- [10] GeeksforGeeks, *Gaussian Elimination*, <https://www.geeksforgeeks.org/gaussian-elimination/>, truy cập ngày 14/04/2026.
- [11] Manim Community Developers, *Manim Community Documentation v0.18.0*, <https://docs.manim.community/>, truy cập ngày 09/04/2026.